PSOs as fast as PSI: Efficient Private Set Operations from Batch-Homomorphic OKVS Decoding

Hyun Ji Kwag*, Junhyuk Kwon*, Changmin Lee*, and Yongha Son‡
* *Korea University, Seoul, Republic of Korea*
Email: {ijnuyh01, kwon76, changminlee}@korea.ac.kr
‡ *Sungshin Women's University, Seoul, Republic of Korea*
Email: yongha.son@sungshin.ac.kr

***Abstract*—Private set operations (PSOs) let two parties compute set-theoretic functionalities on private inputs while revealing nothing beyond the prescribed output. While private set intersection (PSI) has become highly efficient, many other PSOs remain significantly more expensive. The most effective general framework for such tasks is based on reverse private membership test (RPMT), but even state-of-the-art RPMT constructions rely on heavy elliptic-curve-based primitives.**

In this work, we propose a substantially faster RPMT protocol by replacing the elliptic-curve core with RLWE-based one. Our starting point is the Oblivious Key-Value Store (OKVS) based RPMT framework, whose direct adaptation to RLWE is obstructed by the batching structure of RLWE encryption. To address this, we introduce a batching-friendly variant of OKVS together with a homomorphic batched decoding procedure. We believe that this batching-friendly OKVS and its homomorphic decoding process may be of independent interest.

For a set size 2^{20}, our RPMT-based PSO protocols take only about 3 seconds over LAN network and 120-138MB communication, whose running time is comparable to state-of-the-art PSI. Compared to state-of-the-art PSI-Cardinality and PSI-Card-SUM, this is up to $13.0\times$ speedups. Compared to state-of-the-art PSU, this is up to $3.0\times$ smaller communication while achieving comparable computational cost, which results in up to $3.9\times$ faster running time over WAN.

1. Introduction

Private set operations (PSOs) allow two parties to compute set-theoretic functionalities on their private inputs without revealing additional information beyond the prescribed output. The most widely studied example is private set intersection (PSI), but many other functionalities are also natural and useful, such as PSI-Card, threshold-style variants, private set union (PSU), and payload-bearing operations. However, unlike the most basic PSI that has seen remarkable progress in both computation and communication efficiency over the past several years, many other PSOs still exhibit a noticeable performance gap compared with modern PSI protocols, and there is still much room for improvement.

The state-of-the-art generic framework to achieve PSOs (other than PSI) is a reverse private membership test (RPMT) based one, where the RPMT receiver with input set X learns, for each $y \in Y$ held by the RPMT sender, only the membership bit $1(y \in X)$. Recent works of Zhang et al. [1] and Chen et al. [2] proposed RPMT constructions that greatly improved the efficiency of PSOs. Nevertheless, their underlying operations are based on heavy elliptic curve operations, whose cost remains significantly high. This leaves open the question of whether one can retain the generality of the RPMT paradigm while replacing its expensive core with a substantially lighter algebraic backend.

About During Execution Leakage. Some works [3] argued that RPMT-based PSU has a weak point that the receiver learns some information before the protocol terminates, referred to as *during-execution leakage* (DEL). This motivated *enhanced PSU* functionalities and corresponding protocols [3], [4], typically with some efficiency degradation compared with RPMT-based constructions [1], [2]. Recently, however, [5] proved that, in the semi-honest setting, the enhanced PSU functionality that was designed to prevent DEL turns out to be equivalent to the original PSU functionality. This is the up-to-date fact about DEL to the best of our knowledge, which implies RPMT-based PSOs are still secure in the semi-honest setting.[1]

1.1. Our Contributions

Our goal is to replace the expensive elliptic-curve core in prior oblivious key-value store (OKVS) based RPMT with a much lighter ring learning with errors (RLWE) based one. Our starting point is the RPMT framework of Zhang et al. [1], whose core operation is OKVS decoding over ElGamal-encrypted state. The main challenge for adapting their framework with RLWE-based encryption is the batching property of RLWE-based encryption: in RLWE-based encryption, multiple messages are encrypted into a single ciphertext, and this property makes RLWE-based encryption incompatible to the prior idea to build RPMT.

1. Still, our proposal can be easily converted to DEL-free version, and our main body also presents this in Section D.

Batching-friendly homomorphic OKVS decoding. We present *RSB-OKVS*, a batching-friendly variant of Random-Band OKVS [6] tailored to the SIMD layout of RLWE encryption. Concretely, we construct RSB-OKVS by applying a column permutation, equivalently a linear transformation, to the original RB matrix. This preserves the encoding procedure and failure-probability analysis of RB-OKVS, while restructuring the decoding pattern so that it aligns with RLWE batching. By our efficient *sequencing* technique and *homomorphic batched decoding* procedure for RSB-OKVS, we resolve the above problems and allow OKVS decoding to be evaluated efficiently using lightweight slot-wise homomorphic operations without ciphertext rotations.

Implementation and Performance. We implement RPMT-based PSOs derived from our RPMT protocol and evaluate them for input sizes 2^{16}–2^{22} under bandwidths from 10Gbps to 10Mbps, comparing with prior protocols. For PSU, although the runtime of our protocol is about $1.5\times$ slower than the state-of-the-art PSU protocol [7] on LAN, it reduces communication by up to $3.4\times$ and thus performs better on slower networks; on sub-100Mbps networks, it achieves up to $3.06\times$ speedup over [7]. For PSI-Card and PSI-Card-Sum, our protocol achieves the best runtime for almost every input sizes in our tested network environments. Compared with the state-of-the-art protocols [2], our protocols achieves up to $13.4\times$ speed-up over LAN network, with only $1.1-3.1\times$ overheads on the communication. This results in better runtime for all 1Gbps and 100Mbps settings, and large-input 10Mbps settings.

2. Technical Overview

This section provides a technical overview of our new construction of Reverse-PMT (RPMT) of Figure 1, using OKVS and RLWE-batched PKE. The readers are assumed to be familiar with the batching nature (or SIMD packing) and additive homomorphic property of RLWE-PKE, and the random-band matrix based OKVS construction [6], whose review is given in Section 3.

Parameters: Sender $\mathcal{S}$, Receiver $\mathcal{R}$, set sizes n_y and n_x.
Input: $X \subset \{0,1\}^*$ of $\mathcal{R}$, and $Y = \{y_i\} \subset \{0,1\}^*$ of $\mathcal{S}$
Functionality: Return $\boldsymbol{b} = (b_i = \mathbf{1}(y_i \in X))_{i\in[n_y]}$ to $\mathcal{R}$.

Figure 1: Ideal functionality $\mathcal{F}_{\mathsf{RPMT}}$.

2.1. A Review of ZCLZL RPMT Construction

As a base idea, the receiver $\mathcal{R}$ creates $n_x = |X|$ different ciphertexts c_i of some indicator string IS, and OKVS-encodes them by using each x_i as a key and the corresponding ciphertext c_i as its value. If the sender $\mathcal{S}$ decodes this OKVS encoding at $y_j \in Y$ to obtain c_j^*, then, when $y_j \in X$, c_j^* is a ciphertext of IS; otherwise, it is a ciphertext that contains another value than IS. $\mathcal{S}$ finally sends the resulting ciphertexts c_j^* to $\mathcal{R}$, who decrypts them into d_j^*, and sets the membership bit $b_j := \mathbf{1}(\mathsf{IS} = d_j^*)$.

To securely realize the above idea, one needs to consider how to securely decrypt the ciphertext obtained by OKVS decoding. [1] proposed two methods based on SKE and PKE, respectively, and we focus on the PKE version since the SKE version shows very poor performance.[2] Precisely, it re-randomizes the OKVS-decoded ciphertext into another ciphertext, and the $\mathcal{R}$ directly decrypt it: an overview can be found at Figure 2. This procedure is secure for the following reasons:

- If $y_j \in X$, the OKVS-decoded ciphertext c_j^* decrypts to IS. However, if one sends c_j^* as it is, $\mathcal{R}$ can match it with one of its original ciphertexts and identify which x_i is contained in Y. Re-randomization prevents this linkage by refreshing the ciphertext while preserving the plaintext; concretely, this is done by adding an independently generated encryption of 0.
- If $y_j \notin X$, re-randomization does not change the underlying plaintext y_i, so the returned ciphertext may still carry information about y_i. Thus, rigorously speaking, their construction is not a secure realization of RPMT.[3] However, since the real goal of [1] was private set union that aims to reveal $Y - X$ to the receiver, such partial information about the element $y_j \notin X$ is not a leakage at all, and hence this leaky construction was fine.

Input : The receiver $\mathcal{R}$ with $X = \{x_i\}_{i\in[n_x]}$, and the sender $\mathcal{S}$ with $Y = \{y_j\}_{j\in[n_y]}$
Output : $\mathcal{R}$ obtains $\boldsymbol{b} = \{b_i = \mathbf{1}(y_i \in X)\}_{i\in[n_y]}$

1) $\mathcal{R}$ samples a random indicator IS, and encrypts it n_x times to have ciphertexts $\{c_i\}_{i\in[n_x]}$.
2) $\mathcal{R}$ computes the OKVS encoding $\boldsymbol{p}$ of the key-value table $\{(x_i, c_i)\}_{i\in[n_x]}$, and send $\boldsymbol{p}$ to $\mathcal{S}$.
3) $\mathcal{S}$ decodes the OKVS encoding $\boldsymbol{p}$ for each y_j to have a ciphertext c_j^*, re-randomizes c_j^* by adding an encryption of zero, and send back it to $\mathcal{R}$.
4) $\mathcal{R}$ decrypts c_j^*. If the decryption result equals to the indicator string IS, it sets the membership indicator $b_i = 1$, otherwise $b_i = 0$.

Figure 2: An overview of ZCLZL construction for (leaky-)RPMT, based on OKVS and a re-randomizable PKE.

2.2. RLWE-PKE instantiation of ZCLZL

We now start to adapt ZCLZL construction (Figure 2) with the RLWE-based PKE. As a starting obstacle, note that the original construction implicitly assumes that one message corresponds to one ciphertext, which makes it

2. The original paper reported much better performance for SKE than PKE, but it is because it omits very heavy preprocessing timing for generating Beaver triples.

3. The authors were also aware of this, and call this construction under the leaky-VODM and leaky-RPMT.

possible to set the key-value table as $\{(x_i, c_i)\}$. However, RLWE-PKE instantiation immediately confronts a difficulty; it cannot associate one ciphertext to each key x_i, because one RLWE ciphertext contains multiple messages.

Switching OKVS encoding and encryption. We resolve this issue by switching the order of OKVS encoding and encryption. Namely, we first OKVS-encode the key-value table $\{(x_i, \mathsf{IS})\}$, where all x_i are assigned the same value IS, to obtain $\boldsymbol{p}$, and then encrypt the vector $\boldsymbol{p}$. The ZCLZL construction is still valid despite of this switching, since RLWE-PKE is well known to have an additive homomorphic property. More precisely, one can homomorphically perform OKVS decoding on y, equivalently, the inner-product computation $\langle \mathsf{row}(y), \boldsymbol{p} \rangle$ between some sparse vector $\mathsf{row}(y)$ and $\boldsymbol{p}$. Therefore, one obtains the OKVS decoding result in the encrypted state.

In fact, this idea of switching the order of encoding and encryption had already been considered in [8], but with ElGamal PKE. In that case, the homomorphic evaluation of $\langle \mathsf{row}(y), \boldsymbol{p} \rangle$ is very trivial. More specifically, writing $\mathsf{row}(y) = (r_i)$, we have

$$\langle \mathsf{row}(y), \boldsymbol{p} \rangle = \sum_{r_i \neq 0} r_i \cdot p_i, \tag{1}$$

and the sender can simply take the corresponding ElGamal ciphertext of each p_i and multiply it by the coefficient r_i.

However, when encryption is performed with RLWE-PKE, $\boldsymbol{p}$ is encrypted in a batched sense. More precisely, writing $\boldsymbol{p} = (\boldsymbol{p}_1 \,\|\, \cdots \,\|\, \boldsymbol{p}_b)$ for $b = \lceil m/N \rceil$, each length-N vector $\boldsymbol{p}_i$ is encrypted into a single ciphertext c_i. This is incompatible with the entry-wise computation used in ElGamal PKE.

OKVS decoding as a matrix–vector multiplication. Therefore, the main technical problem is to compute the encrypted inner product (1) where $\boldsymbol{p}$ is encrypted in a batched manner. For this, note that the sender after all needs to decode the OKVS *for all* $y \in Y$. In this view, OKVS decoding actually corresponds to a matrix–vector product $\boldsymbol{M} \cdot \boldsymbol{p}$, where the row of $\boldsymbol{M}$ corresponding to y is $\mathsf{row}(y)$ and $\boldsymbol{p}$ is the OKVS encoding vector. At this point, also note that we exclusively consider the random-band (RB) matrix-based OKVS construction of [6], in which each $\mathsf{row}(y)$ consists of a random band—namely, a short sequence of entries of fixed length—starting at a random position.

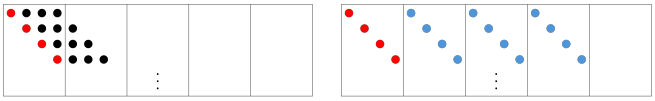
Then, considering that RLWE batching and slot-wise operations are particularly well suited for diagonal matrix–vector multiplication, it is natural to realign the rows of $\boldsymbol{M}$ so that, with respect to their band starting positions, the matrix becomes (almost) diagonal; we can then formulate homomorphic OKVS decoding to a matrix–vector multiplication where the matrix consists of consecutive diagonals.

2.3. Homomorphic Random-band Matrix-Vector Multiplication

The diagonalized random band is more homomorphic-friendly, but two problems remain.

(1) Each row consists of a contiguous random band of length w. A naive matrix–vector multiplication would therefore still be expensive, as it would require w rotated copies of the encoding vector $\boldsymbol{p}$.
(2) In fact, a fully diagonalized matrix is rarely achievable, since the starting position of each random band is determined randomly from the underlying items in X, which will yields collisions between the starting positions with overwhelming probability.

Random spaced bands. Let us address the first problem, while assuming that the second problem (full diagonalization) is resolved somehow. Our idea is to replace each contiguous band by N-spaced entries, where N is the RLWE slot size. Note that if this N-spacing idea works well, the encrypted computation of $\boldsymbol{M} \cdot \boldsymbol{p}$ can be carried out using only slot-wise additive homomorphic operations, without any ciphertext rotations; see Figure 3.



(a) (Diagonalized) Random bands

(b) (Diagonalized) Random spaced bands

Figure 3: Diagonalized matrices. (Each small square represents a $N \times N$ sub-block.)

To make this N-spacing idea more substantial, it should be noted that all prior OKVS-related matrix proposals [6], [9], [10], [11] involve a complicated argument related to encoding/decoding efficiency and failure probability analysis, which are indeed the most complicated part when proposing a new OKVS construction. Therefore, we should provide the same argument for our new spaced matrix family. As a simple solution, we define the *random spaced band* (RSB) matrix as a fixed column-wise permutation of a random-band matrix, where the permutation is determined by a rigorous extension of the N-spacing idea. More precisely, given a key set K, we construct the corresponding random spaced band matrix as in Algorithm 1, where the concrete form of the permutation matrix $\boldsymbol{C}$ is given in the main body as Equation (3).

Algorithm 1 RSB-OKVS.GenMat

Input: A key set $K = \{k_i\}_{i \in [n]}$ and a spacing length N
Output: Spaced random matrix $\hat{\boldsymbol{M}}$
1: Generate a random band matrix $\boldsymbol{M}$ using K
2: **return** $\hat{\boldsymbol{M}} = \boldsymbol{M} \cdot \boldsymbol{C}_N$ where $\boldsymbol{C}_N$ is the N-spacing matrix defined as Equation (3).

Thanks to this column permutation-based definition, encoding with respect to the RSB matrix can be carried out easily by using the efficient encoding algorithm for the RB matrix [6], as in Algorithm 2. Furthermore, the full rank failure-probability analysis for the RB matrix in [6] remains valid for the RSB matrix as well. More precisely, if the row size $m \approx (1 + \varepsilon)n$ and the band width w provide failure

probability $2^{-\lambda}$ for the random-band matrix, then of course the RSB matrix produced by Algorithm 1 does as well. Therefore, the methodology used in [6] to choose (ε, w) (explained in Section F) is still valid in our case, and in fact we set the parameters in this way.

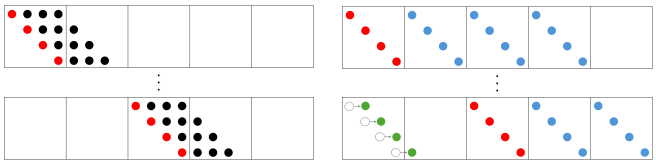
Algorithm 2 RSB-OKVS.Encode

Input: A key-value set $\{(k_i, v_i)\}_{i\in[n]}$, a spacing length N
Output: Encoding $\hat{\boldsymbol{p}}$

1: Generate a random band matrix $\boldsymbol{M}$ using $K = \{k_i\}_{i\in[n]}$
2: Solve the linear system $\boldsymbol{M}\boldsymbol{p} = \boldsymbol{v}$ (using the efficient method of [6])
3: **return** $\hat{\boldsymbol{p}} = \boldsymbol{C}_N^{-1}\boldsymbol{p}$ where $\boldsymbol{C}_N$ is defined as Equation (3).

Remark 1. We do not require the OKVS encoding itself to be secure, because the sender observes its encryption; that is, it suffices that the encryption satisfies IND-CPA security. However, in some OKVS applications, the OKVS encoding is required to be uniformly random whenever the value vector is sampled uniformly; the original ZCLZL RPMT construction [1] and the circuit-PSI construction [12] require this property. For any possible application, we remark that this random encoding property also holds for our RSB matrix also.

As one downside of the column-permutation-based definition, a contiguous random band of length w is shifted by one position each time it crosses a row boundary, as illustrated by the green super-diagonal in Figure 4. Because of this, homomorphic OKVS decoding can no longer be carried out using only the ciphertexts of $\boldsymbol{p}$. More precisely, at most $w-1$ such super-diagonals can arise (when the starting position is placed near the end of a row), and therefore we need at most $w-1$ additional ciphertexts of such shifted copies, or equivalently, $w-1$ homomorphic rotations. Nevertheless, the resulting matrix–vector multiplication is still far more efficient than the original RB-matrix-based computation. More details can be found in Section 4.1.



(a) (Diagonalized) Random bands (b) (Diagonalized) Random spaced bands

Figure 4: Under our RSB matrix definition based on the column permutation, each row boundary cross of the N-spacing band induces one right shift. This is represented as a green super-diagonal in (b).

Matrix sequencing to handle collisions. It remains to address the second issue, namely collisions between band starting positions. The most naive fix would be to build multiple $m \times m$ diagonalized matrices and distribute colliding rows across them, as Figure 5b. However, it can be easily argued that the number of $m \times m$ matrices obtained in this way is $O(\log n)$ [13], which results in up to $O(\log n)$ multiplications of RSB matrix and OKVS encoding vector.[4]

We actually propose a more efficient method by utilizing the RSB matrix structure. Observe that if two rows have the same starting position *modulo* N, then their bands lie on the same diagonal within an $N \times N$ submatrix after diagonalization. Motivated by this observation, we design a procedure that packs the rows according to their *modulo* N starting positions to construct multiple $N \times m$ matrices in a diagonal-block shape; see Figure 5c for an example. We achieve only constant times of computational overhead, compared to the ideally diagonalizable case that involves only one $m \times m$ matrix–vector multiplication. More details can be found in Section 4.2.

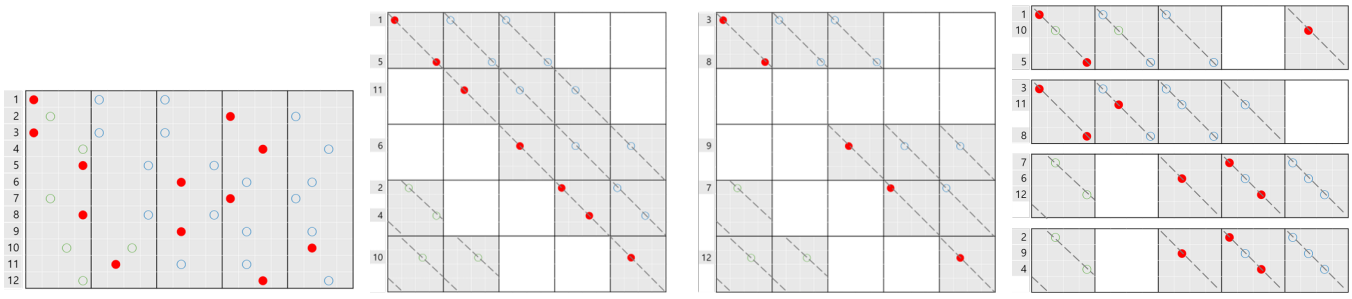
Re-aligning the decryption results. It finally remains a minor issue, because the ideal functionality of RPMT Figure 1 requires the membership bits be aligned in the order of the sender's input elements; $y_j \in Y$. However, if the sequencing produces L many $N \times m$ matrices, the receiver obtains total of $L \cdot N$ decryption results, among which only n_y are effective. For example, in Figure 5c, homomorphic OKVS decoding amounts to multiplying the four matrices shown there by $\boldsymbol{p}$. This leaves four RLWE ciphertexts, and the receiver, after receiving and decrypting them, obtains a total of $4N$ decryption results. To reduce the $L \cdot N$ results into n_y membership bits, we let the sender additionally sends the sequenced position information about which row of which matrix each y_j corresponds to, and let the receiver then realign the $L \cdot N$ accordingly.

As a subtle issue, such sequenced position information may contain some information about Y to the receiver. Such concern could be simply removed using OPRF preprocessing, which is widely used in PSO literature [8], [14], [15], [16] to remedy similar issues: OPRF converts Y into a pseudo-random value (with respect to the receiver), and then the resulting position information at most reveals only such randomized value, which implies nothing about the original items. Furthermore, as a side benefit of this OPRF preprocessing, it also removes the limitation of the leaky-RPMT of Figure 2 for $y \notin X$ case: With OPRF preprocessing, the decrypted value is at most about $F_k(y)$, which is pseudorandom to the receiver and therefore reveals no meaningful information about y.

3. Preliminaries

For $n \in \mathbb{N}$, we denote the set $\{1, 2, \ldots, n\}$ by $[n]$. We use κ, λ to indicate the computational and statistical security parameters, respectively. Vectors are written in lowercase bold letters (e.g., $\boldsymbol{v}$) or by its elements wrapped in parenthesis with the range of index (e.g., $(v_i)_{i\in[n]}$). Matrices are written in uppercase bold letters (e.g., $\boldsymbol{A}$), their elements are

4. For $m = O(n)$, the maximum number of rows that share the same starting position is $O(\log n)$ with high probability by [13].



(a) Original Random Spaced Matrix (b) Diagonal $m \times m$ matrices (c) Block-diagonal $N \times m$ matrices

Figure 5: A toy-example of our matrix sequencing, with $n = 12, m = 15, w = 3$ and $N = 4$. (b) corresponds to the naive diagonalization sequencing, and (c) is an example of our proposed sequencing. Since each nonempty $N \times N$ block translates into one slot-wise multiplication, (b) needs 27 such multiplications, whereas (c) needs only 16.

denoted by subscripts $\boldsymbol{A}_{i,j}$, or sometimes $\boldsymbol{A}[i][j]$. Each i-th row of matrix $\boldsymbol{A}$ is denoted by $\boldsymbol{A}[i]$. We use the indicator notation $\mathbf{1}(S)$, which equals 1 if a statement S is true.

3.1. RLWE-based Encryptions and Batching

We use an encryption scheme whose security is based on the hardness of Ring Learning with Errors (RLWE) [17]. Its message space is $R_p := \mathbb{Z}_p[X]/(X^N+1)$ for some message modulus p and dimension N, and ciphertext space is R_q^2 for some ciphertext modulus q. For a power-of-two N and $p \equiv 1 \pmod{2N}$, the plaintext space is isomorphic to $\mathbb{Z}_p^N$ (as rings), and then one RLWE ciphertext can encrypt multiple integers. We call this the *batching* (or SIMD) property, and write the encryption by $\mathsf{RLWE.Enc} : \boldsymbol{v} \in \mathbb{Z}_p^N \mapsto c \in R_q^2$.

Slot-wise homomorphic operations. RLWE encryption is well-known to be *additively homomorphic*. Under the batching view, one can perform slot-wise arithmetic in the encrypted state: for ciphertexts c_1, c_2 encrypting $\boldsymbol{v}_1, \boldsymbol{v}_2 \in \mathbb{Z}_p^N$, and any plaintext vector $\boldsymbol{v} \in \mathbb{Z}_p^N$, we have

$$\begin{aligned}\mathsf{RLWE.Dec}\left(\mathsf{RLWE.CtxtAdd}(c_1, c_2)\right) &= \boldsymbol{v}_1 + \boldsymbol{v}_2 \in \mathbb{Z}_p^N \\ \mathsf{RLWE.Dec}\left(\mathsf{RLWE.PlainAdd}(c_1, \boldsymbol{v})\right) &= \boldsymbol{v}_1 + \boldsymbol{v} \in \mathbb{Z}_p^N, \\ \mathsf{RLWE.Dec}\left(\mathsf{RLWE.PlainMult}(c_1, \boldsymbol{v})\right) &= \boldsymbol{v}_1 \odot \boldsymbol{v} \in \mathbb{Z}_p^N,\end{aligned}$$

where $\odot$ denotes component-wise multiplication.

Remark 2. RLWE encryption also supports more advanced homomorphic operations, such as ciphertext multiplication and slot rotations, but these are much more expensive than the additive operations used here. Our protocol relies only on additive homomorphic operations, which is a fundamental reason for its good performance.

(Super-)Diagonal matrix-vector multiplications. The slot-wise multiplication can be naturally understood as a diagonal matrix–vector multiplication. More precisely, the multiplication of a diagonal matrix and a vector corresponds to a slot-wise multiplication of the diagonal and the vector. More precisely, it holds that for a diagonal matrix $\boldsymbol{D} = \mathsf{diag}(\boldsymbol{d})$,

$$\boldsymbol{D} \cdot \boldsymbol{p} = \boldsymbol{d} \odot \boldsymbol{p}.$$

For a generalization, consider a k-th super-diagonal matrix that has only nonzero entries d_i on the position $(i, [i+k]_N)$, denoted by $\mathsf{sdiag}_k(\boldsymbol{d})$. Then it holds that

$$\mathsf{sdiag}_k(\boldsymbol{d}) \cdot \boldsymbol{p} = \boldsymbol{d} \odot \rho_k(\boldsymbol{p})$$

where $\rho_k(\cdot)$ is the left rotation by i slots, i.e.,

$$\rho_k : (p_1, \ldots, p_N) \mapsto (p_{1+k}, p_{2+k}, \cdots, p_{N+k})$$

where all indices are taken modulo N into $\{1, 2, \cdots, N\}$.

3.2. Oblivious Key-Value Stores

An oblivious key-value stores [10] is a data structure that compactly represents a desired mapping from a set of keys to corresponding values. The definition is as follows:

Definition 3. An Oblivious Key-Value Stores (OKVS) is parameterized by a key universe $\mathcal{K}$, a field $\mathbb{F}$ and an expansion ratio ε, and consists of the two algorithms:

- $\mathsf{Encode}(T)$: The encode algorithm receives a set of n distinct key-value table $T = \{(k_1, v_1), \ldots, (k_n, v_n)\} \in (\mathcal{K} \times \mathbb{F})^n$ and outputs the encoding $\boldsymbol{p} \in \mathbb{F}^m$ where $m \approx (1+\varepsilon) \cdot n$.
- $\mathsf{Decode}(k, \boldsymbol{p})$: The decode algorithm receives the encoding $\boldsymbol{p} \in \mathbb{F}^m$, a key $k \in \mathcal{K}$ and outputs some value $v \in \mathbb{F}$.

An OKVS is correct if, for any key-value $\{(k_i, v_i)\}_{i \in [n]}$ and $\boldsymbol{p} \leftarrow \mathsf{Encode}(T)$, it holds that $\mathsf{Decode}(k_i, \boldsymbol{p}) = v_i$ for all $i \in [n]$ with overwhelming probability.

Random-band OKVS Construction. State-of-the-art OKVSs [6], [11] are based on solving sparse linear system, and this paper exclusively considers the random-band (RB) matrix based construction [6]. Let $\mathsf{H}_1 : \mathcal{K} \to [m]$ and $\mathsf{H}_2 : \mathcal{K} \to \{0,1\}^w$ for appropriate $w \ll m$ be two random functions. Each key $k \in \mathcal{K}$ is mapped into a sparse vector in $\mathbb{F}^m$ that has only w nonzero components $\mathsf{H}_2(k)$ starting at $\mathsf{H}_1(k)$; this map is written as $\mathsf{row} : \mathcal{K} \to \mathbb{F}^m$. The Encode and Decode algorithms with row map are as follows:

- $\mathsf{Encode}(\{(k_i, v_i)\}_{i \in [n]})$: Solve the linear system

$$\boldsymbol{M} \cdot \boldsymbol{p} = (v_1, \cdots, v_n)^{\mathsf{T}}$$

where $\boldsymbol{M} = [\mathsf{row}(k_1), \cdots, \mathsf{row}(k_n)]^\mathsf{T} \in \mathbb{F}^{n\times m}$, and output the solution $\boldsymbol{p} \in \mathbb{F}^m$. The random-band structure of $\boldsymbol{M}$ enables to solve this linear system in $O(nw)$ time.

- $\mathsf{Decode}(k, \boldsymbol{p})$: Output $\langle \mathsf{row}(k), \boldsymbol{p}\rangle$ over $\mathbb{F}$. The random-band structure reduces this inner product to

$$\sum_{i=1}^{w} u_i \cdot p_{\mathsf{H}_1(k)+i-1}$$

where $\boldsymbol{u} = \mathsf{H}_2(k)$, which takes $O(w)$ time.

For an input size n and expansion ratio ε of $m \approx (1+\varepsilon)n$, the band width w is chosen so that the induced RB matrix $\boldsymbol{M} \in \mathbb{F}^{n\times m}$ has full rank with probability at least $1 - 2^{-\lambda}$, where $\lambda = 40$ is a statistical security parameter. This implies the encoding failure probability less than $2^{-\lambda}$. The details of the (ε, w) parameter-selection methodology of [6] can be found in Section F.

3.3. Private Set Operations from RPMT

Reverse-PMT (Figure 1) yields several private set operations; we describe representative examples below. In all cases, the RPMT phase dominates the overall cost.

PSI-Card. The receiver wants to learn only the intersection size $|X \cap Y|$, while the sender learns nothing. First note that the RPMT receiver can directly obtains $|X \cap Y|$ by taking the Hamming weight of the resulting membership vector. Furthermore, since the membership-indicator vector *aligned by the sender's set* that is not known to the receiver, the receiver cannot know any specific membership of $x_i \in Y$.[5] That is, RPMT directly implies PSI-Card.

PSI-Card-Sum. In addition to $|X \cap Y|$, the receiver learns $\sum_{y\in X\cap Y} d_y$, where $D_Y = \{d_y\}_{y\in Y}$ is the payload set for Y. This can be realized by further performing (1-out-of-2) oblivious transfer (OT). For each y_i, the sender samples $r_i \leftarrow \mathbb{Z}_p$ and sets the OT messages to $(-r_i, -r_i + d_{y_i})$, while the receiver uses b_i as its choice bit. The receiver obtains $m_i = -r_i + \mathbf{1}(y_i \in X) \cdot d_{y_i}$. Letting $R := \sum_{i=1}^{n_y} r_i$ and $M := \sum_{i=1}^{n_y} m_i$, the pair (R, M) is an additive sharing of $\sum_{y\in X\cap Y} d_y$; revealing the shares completes PSI-Card-Sum.

Private Set Union (PSU). The receiver wants to learn $X \cup Y$, equivalent to $Y \backslash X$. This can be realized by performing OT where the sender offers $(y_i, \perp)$ and the receiver uses b_i as its choice bit. Since the element y_i is transferred only when $b_i = 0$, i.e., $y_i \notin X$, $\mathcal{R}$ can end with $Z = Y \backslash X$.

4. RSB Matrices with Homomorphic Decoding

This section proposes a variant of the Random Band Oblivious Key-Value Stores (RB-OKVS) construction [6] that is compatible with RLWE batching, together with a tailored algorithm for efficient homomorphic OKVS decoding. Specifically, given encryptions of the OKVS encoding vector $\boldsymbol{p}$, the algorithm homomorphically evaluates $\boldsymbol{Mp}$ where $\boldsymbol{M}$ is a plain random-band matrix induced by $K = \{k_i\}_{i\in[n]}$. More precisely, to obtain the variant RB matrix compatible with batching, Section 4.1 introduces a matrix family based on *spacing* random bands by the batching slot size N. Then, Section 4.2 addresses collisions among starting positions via a sequencing technique that produces multiple matrices, each of which admits an efficient diagonal-block evaluation. Throughout this section, we assume that the OKVS encoding length m is a multiple of the RLWE slot size N, and we write $b = m/N$.

5. More precisely, this can be achieved by having the sender randomly permute its set and then execute RPMT.

4.1. Random Spaced Band

We propose a variant of RB matrix family that places each random band element *spaced N positions apart* rather than consecutively, what we call random spaced band (RSB) matrix. From this we expect that, given this variant matrix $\boldsymbol{M}$ is diagonalizable with respect to the starting points, each $N \times m$ sub-matrix of M contains only diagonal entries, so that we only need the ciphertexts c_i that encrypts $\boldsymbol{p}_i$ to compute $\boldsymbol{M} \cdot \boldsymbol{p}$. We now describe this more thoroughly.

Definition of a RSB matrix family. The N-spacing idea is realized by the following column remapping:

$$1 \mapsto 1, \qquad 2 \mapsto N+1, \qquad 3 \mapsto 2N+1, \qquad \cdots \tag{2}$$

However, a naive extension of this rule as

$$i \mapsto (i-1)\cdot N + 1 \pmod m$$

no longer defines a one-to-one mapping between columns: for any $i_s \in [N]$, the index $i = (i_s - 1)\cdot b + 1$ satisfies $(i-1)N + 1 \equiv 1 \pmod m$.

To extend the idea in Equation (2) to a column-wise bijection, we consider each column index $i \in [m]$ in its unique representation

$$i = (i_s - 1)\cdot b + i_b, \qquad i_s \in [N],\ i_b \in [b],$$

where $b := m/N$, and define a column-index permutation $\pi_N : [m] \to [m]$ by

$$i = (i_s - 1)\cdot b + i_b \ \longmapsto\ (i_b - 1)\cdot N + i_s.$$

It is immediate to verify that π_N is indeed a permutation.

Using this observation, we define the RSB matrix family as the image of the RB matrix family under this fixed column permutation: given any RB matrix $\boldsymbol{A} \in \mathbb{F}^{n\times m}$, its RSB matrix is $\boldsymbol{B} := \boldsymbol{A} \cdot \boldsymbol{C}_N$, where $\boldsymbol{C}_N \in \{0,1\}^{m\times m}$ is the column-permutation matrix induced by π_N, i.e.,

$$\boldsymbol{C}_N[i][j] = \begin{cases} 1 & \text{if } j = \pi_N(i), \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

As observed in Algorithms 1 and 2, the RSB matrix generation and OKVS encoding with respect to a RSB matrix can be efficiently done using existing methods for RB matrices [6].

Shape of diagonalized RSB matrices. We will perform the matrix–vector multiplication after diagonalizing the starting positions of the RSB matrix, and hence we need to understand the resulting placement of elements precisely. Here, however, a minor issue arises: the column permutation π_N does not move a consecutive band entirely into an N-spaced band. Each time a consecutive band crosses one of the b boundaries, the corresponding spaced band crosses an entire row boundary, and at that point the spaced-band element is placed one position to the right of its natural position modulo m. A toy graphical example of this process is provided in Section B.

As a result, every $N \times m$ submatrix of the diagonalized RSB matrix can have up to $w-1$ non-main diagonals, which in turn implies that we need $w-1$ extra ciphertexts encrypting rotated copies of corresponding parts of $\boldsymbol{p}$; the concrete construction of each such ciphertext is described in detail in Section 4.3. Nonetheless, this is still a substantial improvement from the original RB matrix, where a length-w consecutive band requires w slot-rotated copies of $\boldsymbol{p}$, equivalent to $w \cdot b$ ciphertexts.

4.2. Sequencing

Although we defined the RSB matrix family in a form well suited for diagonal matrix–vector multiplication, this approach is effective only when the matrix can be fully diagonalized. However, because $\mathsf{H}_1(k_i)$ is random, collisions among row starting positions are inevitable. Even a single collision prevents the RSB matrix $\hat{\boldsymbol{M}}$ from being perfectly diagonalized, which in turn undermines our diagonal matrix–vector multiplication strategy. To address this, we present a technique—which we call *sequencing*—as described below.

Naive sequencing into $m \times m$ matrices. A naive remedy is to distribute rows with colliding starting positions across multiple diagonalized $m \times m$ matrices like Figure 5b. The number of matrices produced, denoted L, equals the maximum number of keys mapped to the same starting position, which can be modeled as the maximum load in the standard balls-into-bins process with n balls and m bins. For $m = O(n)$, the maximum load is $O(\log n)$ with high probability (e.g., [13]), so the naive method produces $O(\log n)$ diagonal matrices and incurs $O(\log n)$ multiplicative computational overhead.

One might expect the overhead to be smaller because later matrices become sparse as the number of bins occupying more than i items decreases. However, under our diagonalized matrix–vector evaluation, the cost of $\hat{\boldsymbol{M}}_i \cdot \boldsymbol{p}$ depends on the number of nonzero $N \times N$ blocks (equivalently, nonzero $N \times m$ row-blocks), since each such block contributes at least one diagonal term in the homomorphic evaluation. Thus, entry-wise sparsity does not translate into a proportional reduction in the number of nonzero blocks, and the overall runtime remains close to $O(\log n)$ times an m-dimensional matrix–vector multiplication.

Compact sequencing into $N \times m$ matrices. We avoid the above overhead by exploiting the structure of the RSB matrix $\hat{\boldsymbol{M}}$ as follows. Observe that two rows are aligned to the same diagonal position after diagonalization whenever their starting points are equal modulo N. Using this, we build *multiple* $N \times m$ matrices by grouping each key k_i into the bin indexed by $\mathsf{H}_1(k_i) \bmod N$, i.e., we move from an n-balls-into-m-bins model to an n-balls-into-N-bins model. Since $N \ll n$, the loads become much more balanced, which significantly reduces wasted computation. Moreover, the resulting matrices are well suited for block-level matrix–vector multiplication: the i-th row appears only near the i-th diagonal within the overall matrix structure.

More compact sequencing with clustered rows. The above compact sequencing can be further improved. For homomorphic evaluation, the cost of multiplying $\boldsymbol{M} \in \mathbb{Z}_p^{N \times m}$ and $\boldsymbol{p} \in \mathbb{Z}_p^m$ is governed by the number of nonzero $N \times N$ blocks (equivalently, nonzero $N \times m$ row-blocks). While naive sequencing yields a $N \times m$ matrix with exactly w nonzero diagonals, compact sequencing may scatter row starting points across the entire matrix, potentially increasing the number of nonzero diagonals up to $m/N + w - 1$ for every sequenced matrix.

Motivated by this, we propose Algorithm 3, which reallocates the n rows (with starting points $a_i := \mathsf{H}_1(k_i)$) into a collection of $N \times m$ matrices while keeping the starting points clustered. Concretely, a row starting at a_i is placed in block $b_i := \lceil a_i/N \rceil$ and row $\tau_i := a_i \bmod N$. Given a span limit ζ, we insert each row into an existing $N \times m$ layer matrix as long as its occupied block (i.e., blocks that contain at least one row start) span remains at most ζ after insertion; otherwise, we create a new $N \times m$ layer. Therefore, each sequenced $N \times m$ matrix has at most $w + \zeta$ cyclic diagonals. A toy graphical example of this process is provided in Section B.

Note that we also record the placement information P, whose role is explained later in Section 5.

Choice of span limit ζ. The choice of span parameter ζ creates a trade-off between computation and communication cost. A smaller ζ reduces the number of diagonals $(w + \zeta)$ and hence reduces the computational cost, but it makes the span constraint harder to satisfy and increase the number of layers L, equivalently the number returning ciphertexts. Our rationale for choosing a well-balanced ζ is as follows. First, note that the smallest possible value of L is the maximum bin load in an n-balls-into-N-bins experiment, which is $O(n/N)$ with high probability in our parameter regime of interest, namely $N \ll n$ [13]. Considering this, for each input size n, we choose ζ ensuring that the resulting number of layers L also remains within a small constant factor of n/N, while ensuring ζ to be smaller than w so as not to make the computational cost too large. This enables us to treat $w + \zeta = O(w)$ and $L = O(n/N)$ in the cost analysis later in Section 5.3, and the concrete values are discussed in the performance evaluation in Table 1 of Section 6.

Algorithm 3 RSB-OKVS.Sequencing

Input: RSB-matrix $\hat{\boldsymbol{M}}$ with i-th row starting at a_i, and span limit ζ
Output: Layer matrices $\{\bar{\boldsymbol{M}}_j\}_{j=1}^{L}$ and position list P

$L \leftarrow 0$ ▷ Number of layers
for $i = 1, \ldots, n$ **do**
  $b_i \leftarrow \lceil \frac{a_i}{N} \rceil$ and $\tau_i \leftarrow a_i \pmod N$
  Try to find a layer $\mu \in [L]$ such that
    $-\bar{\boldsymbol{M}}_\mu[\tau_i]$ is zero ▷ τ_i-th row is not occupied
    $-\max(\beta_\mu, b_i) - \min(\alpha_\mu, b_i) \le \zeta$ ▷ At most ζ block span
  if such μ exists **then**
    Update $(\alpha_\mu, \beta_\mu) \leftarrow (\min(\alpha_\mu, b_i), \max(\beta_\mu, b_i))$
  else ▷ Make a new layer matrix
    Set $L \leftarrow L + 1$ and $(\alpha_L, \beta_L) \leftarrow (b_i, b_i)$, and initialize a zero matrix $\bar{\boldsymbol{M}}_L \in \mathbb{Z}_p^{N \times m}$
  end if
  $\bar{\boldsymbol{M}}_\mu[\tau_i] \leftarrow \hat{\boldsymbol{M}}[i]$ and $\mathsf{P} \leftarrow \mathsf{P} \cup \{(\mu, \tau_i)\}$
end for
return $\{\bar{\boldsymbol{M}}_j\}_{j=1}^{L}, \mathsf{P}$

4.3. Full Homomorphic Decoding Procedure

The homomorphic decoding algorithm is obtained by combining Algorithm 1 and Algorithm 3, whose full description is given in Algorithm 4.

We prepare a ciphertext set $\{c_\alpha\}_{\alpha=1}^{b+w-1}$ where $b = \frac{m}{N}$. Write $\boldsymbol{p} = (\boldsymbol{p}_1 \,\|\, \cdots \,\|\, \boldsymbol{p}_b) \in \mathbb{Z}_p^{m(=Nb)}$, where each $\boldsymbol{p}_i \in \mathbb{Z}_p^N$ is a contiguous length-N block. For $i \in [b]$, $j \ge 0$, $jb+i \le b+w-1$, the ciphertexts form as

$$c_{jb+i} = \mathsf{RLWE.Enc}(\rho_j(\boldsymbol{p}_i)),$$

where $\rho_j(\boldsymbol{p}_i)$ denote the blockwise cyclic left shift of $\boldsymbol{p}_i$ by j slots. In other words, the first b ciphertexts encrypt the base blocks $\boldsymbol{p}_1, \ldots, \boldsymbol{p}_b$, the up to $w-1$ additional shifted ciphertexts encrypt the corresponding one-slot cyclic shifts, and so on. At the final shift level, fewer than b ciphertexts may remain, corresponding to an initial prefix of the blocks. In particular, a width-w RSB band starting at block i uses exactly the consecutive ciphertexts $\{c_i, c_{i+1}, \ldots, c_{i+w-1}\}$, which are always contained in $\{c_\alpha\}_{\alpha=1}^{b+w-1}$.

Since each layer matrix $\bar{M}_i$ obtained by sequencing consists of at most $w + \zeta$ nonzero (super-)diagonals, its encrypted multiplication with $\boldsymbol{p}$ can be carried out with $w+\zeta$ homomorphic slot-wise multiplication and addition, given the above ciphertext set $\{c_\alpha\}_{\alpha=1}^{b+w-1}$.

Reason for pre-rotation, rather than HE-rotation. Some HE familiar readers may consider letting the sender generate the additional $w-1$ ciphertexts via homomorphic rotation, rather than sending pre-rotated $w-1$ ciphertexts; this will save the communication, with some additional computational cost for homomorphic rotations. However, our deployed BGV HE [18] scheme do not directly support a full cyclic rotation over all N slots. Rather, the packed slots are viewed as a $2 \times (N/2)$ array, in the sense that native slot manipulations are row/column-wise rotations. This makes the vectorized rotation a bit inefficient, and currently we are not aware of the way to efficiently utilize such structure.

Algorithm 4 RSB-OKVS.HomDecode

Input: A key set K, a spacing length N, a span limit ζ, and ciphertexts $\{c_\alpha\}_{\alpha=1}^{b+w-1}$ of the form $c_{jb+i} = \mathsf{RLWE.Enc}(\rho_j(\boldsymbol{p}_i))$ s.t. $jb + i \le b + w - 1$, where $\boldsymbol{p} = (\boldsymbol{p}_1 \,\|\, \cdots \,\|\, \boldsymbol{p}_b)$
Output: OKVS decoded ciphertexts $\{c_t^*\}_{t=1}^{L}$

$\hat{\boldsymbol{M}} \leftarrow \mathsf{RSB\text{-}OKVS.GenMat}(K, N)$
$((\bar{\boldsymbol{M}}_i)_{i \in [L]}, \mathsf{P}) \leftarrow \mathsf{RSB\text{-}OKVS.Sequencing}(\hat{\boldsymbol{M}}, \zeta)$
for $t = 1, \ldots, L$ **do**
  Compute $c_t^* = \mathsf{RLWE.Enc}(\bar{\boldsymbol{M}}_t \cdot \boldsymbol{p})$ using $w + \zeta$ PlainMult and CtxtAdd.
end for
return $\{c_t^*\}_{t=1}^{L}, \mathsf{P}$

Optimization with full-field random bands. A key factor of the high performance of RB-OKVS [6] was that its band elements are binary $\{0, 1\}$. Then, setting the OKVS value field as a characteristic-two field (e.g., $\mathbb{F} = \mathrm{GF}(2^\ell)$), the OKVS encoding (Gaussian elimination on $\boldsymbol{M}\boldsymbol{p} = \boldsymbol{v}$) can be implemented only with row-wise XORs that can be very efficiently performed by modern processors. However, in our setting, we need to work over the prime field $\mathbb{Z}_p$, the plaintext space of RLWE encryption. Then, even though we set the OKVS matrix $\boldsymbol{M}$ as binary, the OKVS encoding must be performed over $\mathbb{Z}_p$, so the base computation unit becomes modular arithmetic (and potentially inversions) rather than XORs. Thus, binary band elements offer little benefit over $\mathbb{Z}_p$.

Instead, we rather come up with an optimization that samples band elements from the full field $\mathbb{Z}_p$. The underlying rationale is as follows. In [6], the band-width w (and expansion rate ε) is chosen so the induced RB matrix is full rank with probability at least $1 - 2^{-\lambda}$ (and the same parameter provides the same full rank probability for our RSB matrix). In this respect, we can expect that the full field bands would increase the entropy of matrix relative to binary bands, and hence make full rank more likely. In other words, we can expect that the target success probability $1 - 2^{-\lambda}$ can be achieved with a smaller band width w for the full field band, compared to the binary band setting. Indeed, for various n and expansion ratio ε, we observed that the same target probability can be attained with smaller band width; the detailed experimental results are presented in Section F. Since both computational and communication cost of our homomorphic decoding procedure (Algorithm 4) depends on the band width w, this directly improves the performance.

5. Our RPMT protocol

Given the RSB-OKVS and its homomorphic batched decoding, the receiver then could obtain the membership bits by decrypting the RLWE ciphertexts output: it sets $b_i = 1$ if and only if the plaintext equals the indicator string

IS, and $b_i = 0$ otherwise. However, for the sender's set $Y = \{y_j\}_{j\in[n_y]}$, the goal of the RPMT protocol is to let the receiver learn the membership bits in $\{0,1\}^{n_y}$ *aligned with the order of the y_j's*. However, because of the sequencing procedure introduced to produce batching-friendly diagonal matrices, RSB-OKVS.HomDecode (Algorithm 4) outputs L ciphertexts with N slots each, yielding a total of $L \cdot N$ decrypted values, and the order of the y_i's is also changed. Therefore, an additional process is needed so that, among these $L \cdot N$ values, the receiver obtains the membership bits corresponding to Y in the order of $Y = \{y_i\}_{i\in[n_y]}$.

In this section, we address these issues and present a full RPMT construction based on our RSB-OKVS framework. Section 5.1 explains how to extract aligned membership bits from the sequenced outputs and how to handle the associated communication and leakage concerns. Section 5.2 presents the full RPMT protocol, and Section 5.3 discusses concrete parameter choices and their impact on efficiency.

5.1. Generating Membership Indicators

A simple solution to the problem of extracting the valid indicators among $L \cdot N$ ones is as follows. Observe that, while running the sequencing algorithm, the sender records, for each $i \in [n_y]$, the matrix index $\mu_i \in [L]$ and the row index $\tau_i \in [N]$ to which y_i is assigned, in the ordered position list $\mathsf{P} := \{(\mu_i, \tau_i)\}_{i=1}^{n_y}$. Therefore, the sender can simply send this information to the receiver, so that the receiver can extract the valid membership bits in order. More precisely, for each $i \in [n_y]$, the receiver reads the element (that is, IS or a random value) located at position (μ_i, τ_i) among the $N \cdot L$ decrypted results. Arranging these extracted elements in the order of the list P, equivalently in the original order of $Y = (y_1, \ldots, y_{n_y})$, yields the membership bits in $\{0,1\}^{n_y}$.

In fact, the position information $\mathsf{P} = \{(\mu_i, \tau_i)\}_{i\in[n_y]}$ incurs communication linear in n_y, which occupies non-trivial portion of communication cost. Since the receiver only needs to identify the n_y valid entries, we apply an optimization that achieves the same goal with much smaller communication, which is also reflected in our implementation and experimental results. However, to maintain the protocol description simple, we defer the details to Section C.

OPRF preprocessing for security. As a subtle security issue, this position list P implicitly contains partial information about Y. Since, in the RPMT protocol, the receiver is not allowed to learn any information about Y other than the membership bits corresponding to the elements $y_i \in Y$, this is, strictly speaking, a form of leakage.

Fortunately, the PSO literature contains a simple way to address similar issues, namely, randomizing the items using an oblivious pseudorandom function (OPRF), which was first introduced in [14] and later applied in several works [4], [16], [19]. More precisely, it suffices to preprocess X and Y into their PRF values using an OPRF, where the receiver plays the OPRF receiver role (see Figure 9 for the ideal functionality). Then, from the position list P, the receiver learns at most information about the randomized values, which resolves the above security issue.

As another security benefit, OPRF preprocessing also resolves a different issue. Recall from Section 2.1 that, in the original ZCLZL construction, when $y \notin X$, the decryption result could reveal information about the item y (thus the construction could realize only leaky-version of RPMT). However, after OPRF preprocessing, the decrypted value retains at most information about the randomized value. Thus, the decrypted value no longer reveals any meaningful information about y, and the protocol realizes full RPMT.

5.2. Full RPMT protocol

Building on our RSB-OKVS construction and the efficiency/security considerations above, we now present our full RPMT protocol(Figure 6).

Theorem 4. The protocol of Figure 6 securely realizes $\mathcal{F}_{\mathsf{rpmt}}$ of Figure 1 against semi-honest adversaries in the $\mathcal{F}_{\mathsf{oprf}}$-hybrid model, assuming IND-CPA security of the underlying RLWE encryption.

The proof is almost similar to that of [1], except that one additionally needs an IND-CPA argument due to the switching of encoding and encryption, as well as a security argument for the randomization effect of OPRF preprocessing. We only provide an intuitive sketch here, and defer the full proof to Section E.

proof (sketch). First, since we work in the $\mathcal{F}_{\mathsf{oprf}}$-hybrid model, the two parties learn nothing from the OPRF phase beyond its prescribed outputs.

An adversarial sender receives only the RLWE ciphertexts of $\boldsymbol{p}$. By the IND-CPA security assumption, these ciphertexts are computationally indistinguishable from encryptions of a random vector, and hence reveal no information about the underlying OKVS encoding $\boldsymbol{p}$.

An adversarial receiver receives the OKVS-decoded ciphertexts and the sequencing positions P for $Y' = F_k(Y)$. As discussed in Section 2.1, when $y \in X$, the corresponding decryption result reveals nothing beyond the indicator string IS, thanks to re-randomization. When $y \notin X$, the corresponding decryption result contains at most information about $F_k(y)$; since $y \notin X$, this value is computationally indistinguishable from a random string from the receiver's point of view, and therefore reveals no information. Finally, the sequencing positions P also reveal at most information about $F_k(Y)$, and again, for $y \notin X$, the value $F_k(y)$ is indistinguishable from random. Hence the adversarial receiver learns nothing additional from P either. □

Optimizations from RLWE details. In a standard RLWE-based encryption scheme, a ciphertext $c = (c_0, c_1) \in R_q^2$ consists of two polynomials, so its bit-length is about $2N \log q$ bits for ring dimension N. We apply two optimizations from RLWE literature, which reduces the communication almost half. These optimizations are not described in Figure 6 for the sake of simplicity, but indeed applied for experimental results in Section 6.

Input : $\mathcal{S}$ holds $Y = \{y_j\}_{j\in[n_y]}$ and $\mathcal{R}$ holds $X = \{x_i\}_{i\in[n_x]}$.
Output : $\mathcal{R}$ obtains $\{b_{\mu,i}\}_{\mu\in[L],i\in[N]}$ for $b_{\mu,i}$ whose index (μ, i) is in P.

1) **Setup.**
   - The parties agree on RLWE parameters (N, q), and plaintext modulus $\log p \geq \lceil \lambda + \log n_y \rceil$, and then RSB-OKVS parameters w, m s.t. $b := m/N \in \mathbb{Z}$, and a sequencing span parameter ζ.
   - $\mathcal{R}$ generates a RLWE-PKE key pair $(\mathsf{pk}, \mathsf{sk})$, and sends pk to $\mathcal{S}$.

2) **OPRF preprocessing.**
   - Two parties run $\mathcal{F}_{\mathsf{oprf}}$ on $\mathcal{R}$'s input X and $\mathcal{S}$'s (random) PRF key k.
   - As a result of $\mathcal{F}_{\mathsf{oprf}}$, $\mathcal{R}$ obtains $X' = \{x'_i := F(k, x_i)\}_{i\in[n_x]}$. After then, $\mathcal{S}$ locally computes $Y' = \{y'_j := F(k, y_j)\}_{j\in[n_y]}$.

3) **RSB-OKVS encoding, and encryption.**
   - $\mathcal{R}$ samples an indicator $\mathsf{IS} \leftarrow \mathbb{Z}_p$ and computes $\boldsymbol{p} \in \mathbb{Z}_p^m$ via $\mathsf{RSB\text{-}OKVS.Encode}\left(\{(x'_i, \mathsf{IS})\}_{i\in[n_x]}\right)$
   - $\mathcal{R}$ partitions $\boldsymbol{p} = (\boldsymbol{p}_1 \,\|\, \cdots \,\|\, \boldsymbol{p}_b)$ with $\boldsymbol{p}_i \in \mathbb{Z}_p^N$, and generates the ciphertext set $\mathcal{C} = \{c_\alpha\}_{\alpha=1}^{b+w-1}$ of the form $c_{jb+i} = \mathsf{RLWE.Enc}\big(\rho_j(\boldsymbol{p}_i)\big)$ s.t. $i \in [b]$, and $jb + i \leq b + w - 1$.
   - $\mathcal{R}$ sends $\mathcal{C} = \{c_\alpha\}_{\alpha=1}^{b+w-1}$ to $\mathcal{S}$.

4) **Homomorphic RSB-OKVS decoding, and decryption.**
   - $\mathcal{S}$ gets decoded ciphertexts $\{c_\mu\}_{\mu\in[L]}$ and sequencing position P via $\mathsf{RSB\text{-}OKVS.HomDecode}(Y', N, \zeta, \mathcal{C})$ (Algorithm 4).
   - For each $\mu \in [L]$, $\mathcal{S}$ re-randomizes c_μ, by computing $c_\mu^{(0)} \leftarrow \mathsf{RLWE.Enc}(\mathbf{0})$[a] using pk and setting $\widetilde{c}_\mu \leftarrow \mathsf{RLWE.CtxtAdd}(c_\mu, c_\mu^{(0)})$.
   - $\mathcal{S}$ sends back $\{\widetilde{c}_\mu\}_{\mu\in[L]}$ to $\mathcal{R}$, who obtains $\boldsymbol{d}_\mu = (d_{\mu,i})_{i\in[N]} = \mathsf{RLWE.Dec}(\widetilde{c}_\mu) \in \mathbb{Z}_p^N$.
   - For each $\mu \in [L]$ and $i \in [N]$, $\mathcal{R}$ defines $b_{\mu,i} := \mathbf{1}(d_{\mu,i} = \mathsf{IS}) \in \{0, 1\}$.

5) **Extract and re-align n_y membership indicators.**
   - $\mathcal{S}$ further sends the sequencing position information $\mathsf{P} = \{\mathsf{pos}(y'_j) = (\mu_j, \tau_j)\}_{j\in[n_y]}$ to $\mathcal{R}$.[b]
   - $\mathcal{R}$ leaves n_y elements among $\{b_{\mu,i}\}_{\mu\in[L],i\in[N]}$ by extracting $b_{\mu,i}$ whose index (μ, i) is in P.

a. Should retain sufficiently large noise term, for noise-flooding.
b. In the real implementation, we do not transmit P explicitly; instead we apply communication-saving optimization described in Section C.

Figure 6: Our RPMT protocol

First, when the receiver sends encryptions of the encoding vector to the sender, the first component c_0 of each $(c_0, c_1) \in R_q^2$ can be compressed into with a short seed. The sender can locally expand this seed to reconstruct c_0, so only the seed and c_1 need to be sent.

Second, when the sender returns the decoded ciphertexts, seed compression is no longer applicable because these ciphertexts have already undergone homomorphic operations. Instead, as no further homomorphic computation is needed, the receiver can switch down the ciphertext modulus to reduce the size of the returned ciphertexts.

Extension to secret-shared PMT. The PSO applications such as PSI-Card-Sum and PSU can be obtained directly from RPMT. In some applications, however, it is useful to obtain the membership-test results in a secret-shared state, which is dubbed as secret-shared PMT (ssPMT); see Figure 12 for a rigorous definition. For example, ssPMT can achieve PSI-Threshold that reveals only whether $|X \cap Y|$ exceeds a threshold t, or PSI-Sum without revealing the intersection cardinality. In addition, the construction of DEL-free PSU protocols [3], [4] is essentially keeping the membership-test results secret-shared until the final phase.[6]

As a nice feature of our RPMT construction, it can be converted into ssPMT with only small overhead; due to space limitations, we describe this extension in Section D.

5.3. Cost Analysis

This section details our parameter instantiation and analyzes its implications for correctness and efficiency.

Parameter setting for correctness. The indicator bit-length ℓ_{IS} directly affects correctness: if it is too short, a random decoding result for an element $y_i \notin X$ may match IS, causing a false positive. Since a single-element collision occurs with probability $2^{-\ell_{\mathsf{IS}}}$, applying the union bound over

6. The protocols of [3], [4] additionally require a shuffling step since they follow a cuckoo-hashing framework. Our construction does not rely on cuckoo hashing, and hence does not need such a shuffling step.

all $y_i \in Y$ motivates setting $\ell_{\mathsf{IS}} = \lambda + \lceil \log n_y \rceil$, so that $\Pr[\mathsf{false_match}] \leq 2^{-\lambda}$.

The OKVS value field is ℓ_{IS} bits, so we require RLWE plaintext modulus satisfying $\log p \geq \ell_{\mathsf{IS}}$. We choose the RLWE ciphertext modulus q to support depth-1 $\mathsf{PlainMult}$, namely $\log q \gtrsim 2 \cdot \log p$.[7] Finally, we select the batching slot size (equivalently, the RLWE ring dimension) N according to standard RLWE security guidelines [20] for the chosen modulus q and the target security level (e.g., 128-bit security). Concretely, we use $N = 2^{13}$ for all set sizes n.

Asymptotic cost analysis. Our RPMT protocol (Figure 6) has $O(n)$ computation and communication complexity in the input size n (where $n = n_x = n_y$). Due to the space limit, rigorous details can be found in Section G, and we briefly explain the idea. The main reason is that RLWE batching packs N values into one ciphertext: the receiver sends $O(n/N)$ ciphertexts, the sender returns $O(n/N)$ ciphertexts, and each ciphertext has size $O(N)$. Moreover, homomorphic decoding performs $O(n/N)$ layer computations, each using only $O(w)$ slot-wise operations, where $w = O(\lambda)$. Thus, for fixed security parameters, both communication and computation are linear in n.

6. Experimental Results

In this section, we report experimental results for our RPMT protocol, and its extensions to several private set operations, PSU, PSI-Card, and PSI-Card-Sum and compare our scheme with state-of-the-art PSO protocols. All parameters are chosen to ensure $\lambda = 40$ bits of statistical security and $\kappa = 128$ bits of computational security, which is a common choice of literature.

Experiment environments. Our experiment is conducted on a workstation equipped with AMD Ryzen Threadripper 9970X (4.0GHz) CPU and 64GB RAM. The network environment is simulated by localhost on the same machine while the traffic is controlled by the Linux `tc` command. Each party runs over a single thread, which is the same setting to the previous works.

Implementation details. Our implementation in C++ is available at [21]. For the underlying RLWE-based AHE primitive, we rely on SEAL library [22] with HEXL [23] AVX-512 acceleration. We also adapt the public RB-OKVS implementation [24] to implement our RSB-OKVS; furthermore, our PSO protocols need OPRF as a preprocessing step, and we implement the OKVS-based OPRF protocol due to [11] using RB-OKVS. Finally, we rely on `libOTe` [25] for OT-extension backend.

6.1. Concrete RSB matrices Parameter Choices

Our RPMT protocol involves several parameters. We explain the concrete choices summarized in Table 1.

7. In HE, evaluating a depth-d circuit typically requires $\log q \approx (d+1) \cdot \log p$.

TABLE 1: Our deployed parameters for RSB-OKVS: ε is an OKVS row expansion ratio, w is a band width, ζ is a span limit for the matrix sequencing. L is the mean of the number of sequenced layers (equals to the number of returning ciphertexts from $\mathcal{S}$ to $\mathcal{R}$), which is measured by taking average over 100 independent random instances. It can be checked that $w + \zeta = O(w)$ and $L = O(n/N)$, as analyzed in Section 5.3.

N	n	ε	w	ζ	L	n/N
2^{13}	2^{16}	1.8	23	9	22.5	8
	2^{18}	1.2	28	13	62.3	32
	2^{20}	1.1	31	20	198.2	128
	2^{22}	0.7	45	30	665.6	512

First, recall the batching slot size, namely the RLWE ring dimension N, is fixed to $N = 2^{13}$ in all experiments, which is the smallest choice that securely supports our homomorphic OKVS decoding procedure. We then consider the band expansion ratio ε and the band width w. Recall that the requirement for (ε, w) for our RSB matrix is exactly the same as in the original RB matrix [6]: One should choose them so that the encoding failure probability is $2^{-\lambda}$. However, once the additional features and optimizations specific to our protocol are taken into account, the parameter pairs (ε, w) suggested in [6] are no longer optimal in our setting. Therefore, while we follow the methodology of [6], we select (ε, w) differently from their choices.

Basically, the choice of (ε, w) involves the following trade-off. Since the OKVS encoding length is $m \approx (1+\varepsilon)n$, increasing ε directly increases the communication cost for the fresh ciphertexts sending from $\mathcal{R}$ to $\mathcal{S}$. On the other hand, as the rows become longer, the width w required for the R(S)B matrix to be full rank becomes smaller, which reduces the computational cost. However, in our protocol, the number of fresh ciphertexts is $m/N + w - 1$. That is, the communication cost is also affected by w. Consequently, up to a certain point, increasing ε can be expected to reduce both communication and computation at the same time. The parameters in Table 1 are chosen as balancing one, from internal experiments. Naturally, different parameter choices may be optimal in different environments, and some use cases may require a different computation–communication trade-off; we provide relevant data in Section F.

Lastly, as explained in Section 4.2, we choose ζ so as to give a good balance between the number of diagonals $w + \zeta$ in each layer matrix and the resulting number of layers L. In particular, it can be checked that our chosen values of ζ satisfy $w + \zeta = O(w)$ and $L = O(n/N)$.

6.2. Performance Evaluations

This section provides a concrete performance evaluation, and comparison with recent PSO protocols. We consider four network environments: (1) 10Gbps with 0.2 ms latency, (2) 1Gbps bandwidth with 80 ms latency, (3) 100 Mbps bandwidth with 80ms latency, and (4) 10 Mbps bandwidth

with 80 ms latency. The first setting will be referred as LAN network, and the others are as WAN networks.

No setup (offline) timing separation. Before presenting the performance comparisons, we stress that many recent PSO works involve the generation of correlated randomness, especially DEL-free ones [3], [4]. A notable feature of such generation is that it can be performed before the input sets are determined. For this reason, some works separate a setup (or offline) phase, which includes such randomness generation, from the online protocol execution time.

In fact, however, such a separation can be misleading. The setup phase in PSO context is not a one-time cost that can be amortized over many online executions. Rather, in PSO protocols, most of this setup material—in particular, the correlated randomness mentioned above—is consumed in each execution and therefore must be generated again for every run. Under this view, we believe that reporting the full execution cost without separating setup and online phases is more appropriate, and all performance results below follow this convention.

PSU comparisons. Table 2 provides a comparison with recent PSU protocols [1], [2], [7], where all numbers are obtained from running the public implementations,[8] on our machine. Other recent (DEL-free) protocols such as [3] and [4] are omitted, as their performance are already incompetitive. Our protocol achieves a running time comparable to that of [7], which was previously the fastest in the LAN setting, while also attaining communication cost comparable to that of [2], which has the smallest communication among the compared protocols.

One feature not represented in the table is that [7] requires $O(\log n)$ interaction rounds, since it is built by repeatedly invoking a subprotocol that progressively recovers the set difference by utilizing a special data-structure called IBLT (Invertible bloom look-up table). According to the analysis in [7], for large inputs such as $n \geq 2^{20}$, this leads to dozens of communication rounds. In contrast, our protocol is constant-round throughout: the OPRF we deploy [6] is constant-round, the RPMT phase consists of a single batched transmission of RLWE ciphertexts and a single response, and the final OT is also constant-round. Because of this difference, [7] degrades much more sharply as latency increases—compare the LAN and 1Gbps WAN settings—in addition to the degradation caused by its larger communication cost.

Another notable point is that, except for the final OT, RPMT-based PSU protocols including ours depend only weakly on the actual item length, and therefore their communication increases only mildly as the item length grows. More precisely, when the items are long, it is standard to hash them to a shorter length sufficient to avoid collisions—namely, about $\lambda + \log(n_x n_y)$ bits—and then run the OPRF and RPMT phases on those hashed values.

8. [1]: https://github.com/alibaba-edu/mpc4j
[2]: https://github.com/yuchen1024/Kunlun
[7]: https://github.com/asu-crypto/IBLT-based-PSU

In contrast, [7] need to build IBLT on the set itself as input, which makes it difficult to apply a pre-hashing technique. Thus, its total communication cost strict-linearly depends on the bit-length of the input items. Concretely, for $L = 256$ bits item (such as SHA outputs), [7] communication increases to 1338.07 MB, whereas ours becomes 188.51 MB. This gap naturally widens for longer L, and Figure 7 visualizes the trend. Note that, as the original paper [7] provides no closed formula for communication cost, we compute it by ourselves in Section H.

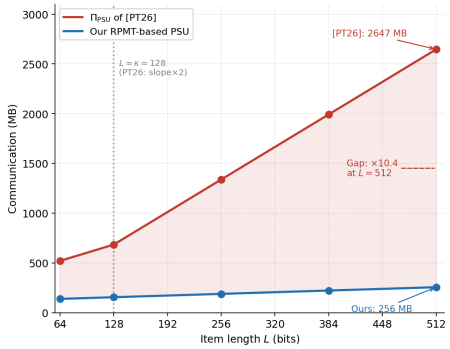


Figure 7: Comparison of communication cost with respect to item length, for a fixed set size $n = 2^{20}$

In summary, while [7] remains best on LAN, our protocol generally performs best at lower bandwidths and is therefore expected to be faster in typical settings, where the worldwide median internet speed is about 100 Mbps [26].

Remark 5. Although [7] targets only PSU, one may wonder whether their idea extends to other PSOs and could achieve similar performance there as well. Regarding, the original paper only gives the following open-ended remark: *"We believe it may also have broader applications in secure set operations ⋯"*, so it seems to be left as an open question.

PSI-Card and PSI-Card-Sum Comparisons. Table 3 provides a comparison with recent PSI-Card and PSI-Card-Sum protocols [2], [28], where all numbers are obtained from running the public implementations again,[9] on our machine. The result shows that our protocol records the fastest running time for almost every setting. Some exception is observed for very low-bandwidth environments such as 10 Mbps, because of our larger communication cost than that of [2]. However, as the input size grows, the computational-efficiency advantage of our protocol accumulates and eventually outweighs this network disadvantage, allowing it to again achieve the fastest running time. Indeed, even at 10 Mbps, our PSI-Card-Sum protocol becomes the fastest from $n \geq 2^{20}$, and our PSI-Card protocol also becomes the fastest at $n = 2^{22}$.

Remark 6. A recent work [29] claimed to have significantly faster PSOs, by proposing an improved OPRF construc-

9. [28]: https://github.com/osu-crypto/PSI-analytics
[2]: https://github.com/yuchen1024/Kunlun

TABLE 2: Comparison of PSU protocols. The item length is 64-bit for every case. The best (and the second) values are marked in red (and blue), resp. All timing reports include setup timing, such as correlated random generation [27].

Set Size	Protocol	Comm. (MB)	Runtime (s)			
			10Gbps	1Gbps	100Mbps	10Mbps
2^{16}	[1]	11.68	3.12	4.09	4.55	12.78
	[2]	6.78	2.47	3.66	3.82	8.26
	[7]	39.09 / 29.74†	0.14	3.07	5.27	26.87
	Ours	16.30	0.24	2.23	3.68	15.72
2^{18}	[1]	46.88	10.69	11.71	13.98	47.31
	[2]	27.07	9.83	11.61	12.97	31.01
	[7]	130.07 / 99.87†	0.53	7.99	13.83	87.87
	Ours	41.34	0.76	3.13	6.20	37.32
2^{20}	[1]	187.76	40.81	43.30	51.30	187.22
	[2]	108.23	39.31	43.41	49.99	127.09
	[7]	520.18 / 409.04†	2.46	27.25	49.66	338.55
	Ours	138.18	3.17	7.02	16.22	121.87
2^{22}	[1]	751.71	170.11	181.59	213.18	748.90
	[2]	432.87	159.74	172.78	199.89	514.25
	[7]	2078.78 / 1633.55†	10.82	103.97	186.30	1344.22
	Ours	473.85	14.44	24.26	61.64	414.32

†: [7] used different parameter sets for LAN and WAN networks, to trade-off between communication and computation cost.

tion. However, this work has the following two serious issues.[10] First, the OPRF reported in the paper does not achieve the claimed security, and fixing this requires at least 4x communication cost for OPRF. Second, their implementation locally generates the Beaver triple, rather than a proper joint protocol between the two parties. As the beaver triple generation time occupies quite a large portion of previous works [11], this is quite unfair. For these reasons, we do not compare with this work.

Breakdown of our RPMT. Table 4 reports the runtime and communication breakdown of our RPMT protocol for $n = 2^{20}$ in the LAN setting. As one can easily expect, the largest computational bottleneck is RSB-OKVS.HomDecode, which is consistent with the cost analysis: each sequenced layer requires $w+\zeta$ homomorphic operations, and the total work scales as $O(nw)$.

7. Conclusion

This paper proposes a RPMT protocol based on RLWE-batching friendly OKVS and their homomorphic decoding, and shows remarkable performance improvement for almost every PSOs previously discussed. However, our approach is not the optimal solution for every variant of PSO. While standard PSI protocol is technically feasible within our framework, existing protocols based on a single OPRF execution [6], [11] remain the most efficient solutions for PSI protocol. Furthermore, regarding the computation of the inner product over the intersection—where both parties possess associated payloads—several dedicated solutions exist [30], [31]. Extending our approach to efficiently support this functionality remains an open problem.

10. The authors also acknowledged this by personal communication, and the ePrint version is currently withdrawn.

TABLE 3: Comparison of PSI-Card and PSI-Card-Sum protocols. The best (and the second) values are marked in red (and blue), resp. All timing reports include setup timing, such as correlated random generation [27].

PSI-Card

Set Size	Protocol	Comm. (MB)	Runtime (s)			
			10Gbps	1Gbps	100Mbps	10Mbps
2^{16}	[2]	4.67	2.45	3.31	3.46	6.37
	[28]	36.26	3.03	7.99	10.16	35.82
	Ours	14.52	0.20	1.86	2.88	13.57
2^{18}	[2]	18.66	9.80	11.12	11.95	23.99
	[28]	155.57	11.40	19.68	29.25	141.55
	Ours	37.67	0.73	2.70	5.30	33.81
2^{20}	[2]	74.66	39.20	42.14	46.59	99.18
	[28]	664.87	47.76	69.81	111.24	595.85
	Ours	120.53	3.01	6.01	13.99	105.74
2^{22}	[2]	298.63	158.97	168.52	187.18	404.55
	[28]	4733.55	96.13	228.88	526.74	3987.19
	Ours	409.75	14.06	21.62	53.90	356.40

PSI-Card-Sum

Set Size	Protocol	Comm. (MB)	Runtime (s)			
			10Gbps	1Gbps	100Mbps	10Mbps
2^{16}	[2]	6.26	2.55	3.73	3.92	8.01
	[28]	40.61	4.13	12.74	13.32	41.72
	Ours	15.55	0.24	2.23	3.58	15.07
2^{18}	[2]	24.98	10.16	11.96	13.10	29.50
	[28]	172.90	15.81	29.13	34.96	159.57
	Ours	38.35	0.76	3.08	5.88	34.79
2^{20}	[2]	99.85	40.56	44.53	50.53	121.18
	[28]	734.13	65.18	86.70	129.95	661.84
	Ours	126.19	3.16	6.70	15.06	111.15
2^{22}	[2]	399.32	162.14	177.03	202.20	490.30
	[28]	5010.56	167.19	288.87	590.72	4250.07
	Ours	425.85	14.39	22.86	57.01	372.41

TABLE 4: Breakdown of our RPMT over LAN, for $n = 2^{20}$.

Stages	Runtime (s)	Comm. (MB)
OPRF	0.419	17.60
RSB-OKVS.Encode	0.625	-
RLWE.Enc	0.269	55.02
RSB-OKVS.HomDecode	1.626	48.81
RLWE.Dec	0.032	-

References

[1] C. Zhang, Y. Chen, W. Liu, M. Zhang, and D. Lin, "Linear private set union from multi-query reverse private membership test," in *USENIX Security 2023*, J. A. Calandrino and C. Troncoso, Eds. USENIX Association, 2023, pp. 337–354.

[2] Y. Chen, M. Zhang, C. Zhang, M. Dong, and W. Liu, "Private set operations from multi-query reverse private membership test," in *PKC 2024 Part III*, ser. Lecture Notes in Computer Science, Q. Tang and V. Teague, Eds., vol. 14603. Springer, 2024, pp. 387–416.

[3] Y. Jia, S. Sun, H. Zhou, and D. Gu, "Scalable private set union, with stronger security," in *USENIX Security 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024.

[4] B. Tu, Y. Bai, C. Zhang, Y. Cao, and Y. Chen, "Fast enhanced private set union in the balanced and unbalanced scenarios," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 3437–3456. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/tu

[5] S. Pu, J. Gao, and N. Trieu, "Malicious private set union with two-sided output," in *EUROCRYPT 2026*. Springer, 2026.

[6] A. Bienstock, S. Patel, J. Y. Seo, and K. Yeo, "Near-optimal oblivious key-value stores for efficient PSI, PSU and volume-hiding multi-maps," in *USENIX Security 2023*, J. A. Calandrino and C. Troncoso, Eds. USENIX Association, 2023, pp. 301–318.

[7] L. Piske and N. Trieu, "Is PSI really faster than PSU? achieving efficient PSU with invertible bloom filters," in *EUROCRYPT 2026*. Springer, 2026.

[8] C. Zhang, Y. Chen, W. Liu, L. Peng, M. Hao, A. Wang, and X. Wang, "Unbalanced private set union with reduced computation and communication," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1434–1447. [Online]. Available: https://doi.org/10.1145/3658644.3690308

[9] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "PSI from PaXoS: Fast, malicious private set intersection," in *EUROCRYPT 2020 Part II*, ser. Lecture Notes in Computer Science, A. Canteaut and Y. Ishai, Eds., vol. 12106. Springer, 2020, pp. 739–767.

[10] G. Garimella, B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "Oblivious key-value stores and amplification for private set intersection," in *CRYPTO 2021*. Springer, 2021, pp. 395–425.

[11] S. Raghuraman and P. Rindal, "Blazing fast PSI from improved OKVS and subfield VOLE," in *ACM CCS 2022*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds. ACM, 2022, pp. 2505–2517.

[12] P. Rindal and P. Schoppmann, "VOLE-PSI: fast OPRF and circuit-PSI from vector-OLE," in *EUROCRYPT 2021 Part II*, ser. Lecture Notes in Computer Science, A. Canteaut and F. Standaert, Eds., vol. 12697. Springer, 2021, pp. 901–930.

[13] M. Raab and A. Steger, ""balls into bins"—a simple and tight analysis," in *International Workshop on Randomization and Approximation Techniques in Computer Science*. Springer, 1998, pp. 159–170.

[14] H. Chen, Z. Huang, K. Laine, and P. Rindal, "Labeled PSI from fully homomorphic encryption with malicious security," in *CCS 2018*, 2018, pp. 1223–1237.

[15] K. Cong, R. C. Moreno, M. B. da Gama, W. Dai, I. Iliashenko, K. Laine, and M. Rosenberg, "Labeled psi from homomorphic encryption with reduced computation and communication," *ACM CCS*, 2021.

[16] B. Tu, Y. Chen, Q. Liu, and C. Zhang, "Fast unbalanced private set union from fully homomorphic encryption," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 2959–2973.

[17] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *EUROCRYPT 2010*. Springer, 2010, pp. 1–23.

[18] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," *TOCT*, vol. 6, no. 3, pp. 1–36, 2014.

[19] K. Liu, X. Li, and T. Takagi, "Review the cuckoo hash-based unbalanced private set union: Leakage, fix, and optimization," in *ESORICS 2024 Part II*, ser. Lecture Notes in Computer Science, J. García-Alfaro, R. Kozik, M. Choras, and S. K. Katsikas, Eds., vol. 14983. Springer, 2024, pp. 331–352.

[20] M. Albrecht, M. Chase, H. Chen, J. Ding, S. Goldwasser, S. Gorbunov, S. Halevi, J. Hoffstein, K. Laine, K. Lauter, S. Lokam, D. Micciancio, D. Moody, T. Morrison, A. Sahai, and V. Vaikuntanathan, "Homomorphic Encryption Security Standard," HomomorphicEncryption.org, Toronto, Canada, Tech. Rep., November 2018.

[21] https://github.com/yonghaason/RLWE-OKVS.git.

[22] "Microsoft SEAL (release 3.5)," https://github.com/microsoft/SEAL, Apr. 2020, microsoft Research, Redmond, WA.

[23] F. Boemer, S. Kim, G. Seifu, F. D. de Souza, V. Gopal *et al.*, "Intel HEXL (release 1.2)," https://github.com/intel/hexl, 2021.

[24] "Rb-okvs," Google, accessed: 2025-08-20. [Online]. Available: https://github.com/google/private-membership/tree/main/research/okvs

[25] P. Rindal, "libOTe: A fast, portable, and easy to use oblivious transfer library," 2022. [Online]. Available: https://github.com/osu-crypto/libOTe

[26] Ookla, "Speedtest global index," https://www.speedtest.net/global-index, accessed: 2026-04-16.

[27] S. Raghuraman, P. Rindal, and T. Tanguy, "Expand-convolute codes for pseudorandom correlation generators from LPN," in *CRYPTO 2023 Part IV*, ser. Lecture Notes in Computer Science, H. Handschuh and A. Lysyanskaya, Eds., vol. 14084. Springer, 2023, pp. 602–632.

[28] G. Garimella, P. Mohassel, M. Rosulek, S. Sadeghian, and J. Singh, "Private set operations from oblivious switching," in *PKC 2021 Part II*, ser. Lecture Notes in Computer Science, J. A. Garay, Ed., vol. 12711. Springer, 2021, pp. 591–617.

[29] G. Ling, P. Tang, J. Shan, L. Xiang, and W. Qiu, "Faster than ever: A new lightweight private set intersection and its variants," in *NDSS 2026*, 2026.

[30] B. Pinkas, T. Schneider, O. Tkachenko, and A. Yanai, "Efficient circuit-based PSI with linear communication," in *EUROCRYPT 2019 Part III*, ser. Lecture Notes in Computer Science, Y. Ishai and V. Rijmen, Eds., vol. 11478. Springer, 2019, pp. 122–153.

[31] T. Lepoint, S. Patel, M. Raykova, K. Seth, and N. Trieu, "Private join and compute from PIR with default," in *ASIACRYPT 2021 Part II*, ser. Lecture Notes in Computer Science, M. Tibouchi and H. Wang, Eds., vol. 13091. Springer, 2021, pp. 605–634.

Appendix A. Ideal Functionalities

The ideal functionalities for oblivious transfer, oblivious PRF, and Secret-Shared Private Equality Test are as follows.

Parameters: Sender $\mathcal{S}$, Receiver $\mathcal{R}$, message length ℓ_m
Input: $b \in \{0,1\}$ of $\mathcal{R}$, and $m_0, m_1 \in \{0,1\}^{\ell_m}$ of $\mathcal{S}$
Functionality: Output m_b to $\mathcal{R}$.

Figure 8: (1-out-of-2) Oblivious transfer functionality $\mathcal{F}_{\mathsf{ot}}$.

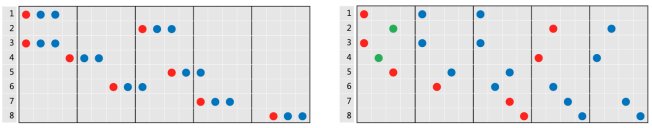
Parameters: Sender $\mathcal{S}$, Receiver $\mathcal{R}$, and a pseudorandom function $F : \{0,1\}^* \times \mathcal{K} \to \{0,1\}^*$.
Input: A PRF key $k \in \mathcal{K}$ from $\mathcal{S}$, and input X from $\mathcal{R}$
Functionality: Output $F_k(X) := \{F(x,k) : x \in X\}$ to $\mathcal{R}$.

Figure 9: Ideal functionality $\mathcal{F}_{\mathsf{oprf}}$ for oblivious PRF.

Appendix B. Generating Homomorphic Decoding Matrix: Example

To illustrate the process in Section 4, we provide the examples as follows. For clarity, we assume small parameter $n = 8, m = 20, w = 3, N = 4$, and $\zeta = 2$.

According to Equation (3), random band (RB) matrix Figure 10a is converted to random spaced band (RSB) matrix Figure 10b. For example, the second row starting point $i = 9$ in RB matrix corresponds to $(i_s, i_b) = (2, 4)$; thus $\pi_N(9) = (4-1) \cdot 4 + 2 = 14$ in RSB matrix.



(a) Random band matrix (b) Random spaced band matrix

Figure 10: Structural comparison between a RB matrix and RSB matrix under the column permutation π_N.

Second, the compact sequencing with clustered rows proceeds as illustrated in Figure 11. The first row (starting position $a_1 = 1$, block index $b_1 = 1$) is placed in the first $N \times m$ layer matrix $\bar{\boldsymbol{M}}_1$, but when processing the second row ($a_2 = 14$, $b_2 = 4$), the block distance exceeds the span limit ($|b_2 - b_1| > \zeta$), so we create a new layer $\bar{\boldsymbol{M}}_2$. Subsequently, the third row ($a_3 = 1$, $b_3 = 1$) is also allocated to a newly created third layer $\bar{\boldsymbol{M}}_3$. This is because it collides with the first row in $\bar{\boldsymbol{M}}_1$, and cannot be placed in $\bar{\boldsymbol{M}}_2$ due to the span limit ($|b_3 - b_2| > \zeta$).

Appendix C. Optimization for Sequencing Positions

Our RPMT protocol, presented in Figure 6, is achieved by additionally transmitting position list $\mathsf{P} = \{\mathsf{pos}(y'_j) = (\mu_j, \tau_j)\}_{j \in [n_y]}$, and sends it to $\mathcal{R}$. However, integers are transmitted using fixed-length encodings (e.g., 32-bit or 64-bit). Hence, directly transmitting $\mathsf{P} = \{\mathsf{pos}(y'_j) = (\mu_j, \tau_j)\}_{j \in [n_y]}$ requires transmitting at least $64 \cdot n_y$-bits, which occupies non-trivial portion in the total transmission.

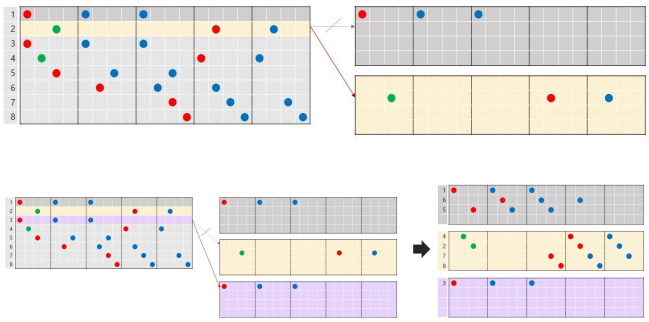


Figure 11: Our compact sequencing process.

To reduce this overhead, we slightly modify the RPMT protocol and then optimize the transmission of the position list. The key observation behind this modification is that what the receiver ultimately needs is the membership bit for each y_i, but not necessarily the result aligned with the original input order $y_1, \ldots, y_{n_y}$. That is, as long as the membership bits are arranged in some common order, both parties can carry out the subsequent computation consistently with respect to that order. To this end, the sender interprets the position list P not in the original element order, but in the lexicographic order of the post-sequencing positions (μ, τ), that is, in the order obtained by listing entries row by row starting from the first row of the first matrix. In other words, the sender reorders $\{y_i\}_{i \in [n_y]}$ according to the lexicographic order of the positions (μ_i, τ_i) to which the elements are assigned. After that, as long as the sender and the receiver refer to the membership bits corresponding to this reordered order in the same way, there is no issue with the correctness of the RPMT protocol. As before, it suffices for the receiver to identify only the valid n_y slots, and for this purpose the sender does not need to transmit the full index pairs (μ_i, τ_i) for all positions. In other words, it is enough to share simpler bit information indicating only whether each row is actually occupied; with this information, the receiver can extract the valid n_y membership bits and recover the same reordered order as the sender. The concrete optimization is described below.

To reduce the communication overhead of this part, we replace it with a more compact encoding. Instead of explicitly sending P, we transmit occupancy bits that indicate only whether a position is occupied along up to L layers (matrices). Concretely, for each $\tau \in [N]$ (row index), we define a bit sequence of length at most L as $(o_{1,\tau}, o_{2,\tau}, \ldots, o_{L,\tau}) \in \{0,1\}^L$, where

$$o_{\mu,\tau} = \begin{cases} 1 & \exists j \in [n_y] \text{ s.t. } \mathsf{pos}(y'_j) = (\mu, \tau), \\ 0 & \text{otherwise.} \end{cases}$$

We then transmit a bitstring obtained by concatenating these per-τ bit sequences in order. This enables the parties to share, using significantly less communication, the same structural information indicating, for each (μ, τ), whether the τ-th row of the μ-th layer is occupied by some element

$y_i \in Y$, without directly sending the integer-pair list P. In other words, we substantially reduce the communication cost by replacing an integer-based encoding with a bit-based encoding.

Using this occupancy information, the sender $\mathcal{S}$ and the receiver $\mathcal{R}$ can agree on the same indices, i.e., the same re-ordering for $y_i \in Y$. $\mathcal{S}$, who performs the sequencing procedure, knows P internally and thus re-orders the n_y elements $\{y_i \in Y\}$ according to the lexicographic order of their positions (μ_j, τ_j). In contrast, the $\mathcal{R}$ does not learn the explicit pairs (μ_j, τ_j); however, from the received occupancy bits $\{o_{\mu,\tau}\}$ it can identify and enumerate exactly those rows for which $o_{\mu,\tau} = 1$, i.e., the rows in which elements $y_i \in Y$ are placed. Hence, $\mathcal{R}$ can enumerate the occupied slots in lexicographic order and extract the corresponding membership bits in the same order used by $\mathcal{S}$.

Appendix D. Extension to Secret-Shared PMT

This section explains how to convert our RPMT protocol into a secret-shared PMT (ssPMT), whose ideal functionality is defined as Figure 12.

Parameters: Sender $\mathcal{S}$, Receiver $\mathcal{R}$, set sizes n_y and n_x.
Input: $X \subset \{0,1\}^*$ of $\mathcal{R}$, and $Y = \{y_i\} \subset \{0,1\}^*$ of $\mathcal{S}$
Functionality: Define $\boldsymbol{b} = (b_i = \mathbf{1}(y_i \in X))_{i\in[n_y]}$. Sample a random $\boldsymbol{r} \leftarrow \{0,1\}^{n_y}$, and return $\boldsymbol{b} \oplus \boldsymbol{r}$ to $\mathcal{R}$, and $\boldsymbol{r}$ to $\mathcal{S}$.

Figure 12: Ideal functionality $\mathcal{F}_{\mathsf{ssPMT}}$.

The key idea is to mask the homomorphically decoded plaintexts before decryption, so that $\mathcal{R}$ cannot locally compare them with the indicator string. The parties then run a secret-shared equality test to obtain shares of the membership bits. More precisely, for each decoded ciphertext c^*_μ (for $\mu \in [L]$), $\mathcal{S}$ samples a random mask $\boldsymbol{r}_\mu \leftarrow \mathbb{Z}_p^N$, computes $\bar{c}^*_\mu \leftarrow \mathsf{RLWE.PlainAdd}(c^*_\mu, \boldsymbol{r}_\mu)$, and sends $\bar{c}^*_\mu$ to $\mathcal{R}$. Then if $y_i \in X$, then decrypted value becomes $d_i = \mathsf{IS}+r_i$ (where IS is the fixed indicator tag, $d_i \in \boldsymbol{d}_\mu$, and $r_i \in \boldsymbol{r}_\mu$); otherwise $d_i = r'_i + r_i$ for uniform r'_i. In $\mathcal{R}$'s view, both cases appear uniformly random, so $\mathcal{R}$ cannot derive membership bits on its own. We remark that this is the core argument of DEL-free PSUs [3], [4] to avoid the during-execution leakage.

Afterward, the parties privately test whether $r_i = d_i - \mathsf{IS}$ and obtain the bit $\mathbf{1}(r_i = d_i - \mathsf{IS})$ in XOR-secret-shared form, i.e., shares $b_{Y,i}$ and $b_{X,i}$ such that $b_{X,i} \oplus b_{Y,i} = b_i$. These secret-shared membership bits (secret-shares of b_i) can be extended to several PSO, by tweaking the OT-based PSO extension of RPMT. For example, given secret-shares of $b_i = b_{X,i} \oplus b_{Y,i}$, $\mathcal{S}$ with $b_{Y,i}$ sets

$$(m_0, m_1) = \begin{cases} (y_i, \perp) & \text{if } b_{Y,i} = 0, \\ (\perp, y_i) & \text{if } b_{Y,i} = 1, \end{cases}$$

and $\mathcal{R}$ chooses with $b_{X,i}$. Then $\mathcal{R}$ obtains y_i iff $b_{X,i} \oplus b_{Y,i} = 0$, i.e., iff $y_i \in Y \setminus X$.

The above transformation additionally requires L plaintext-addition operations and one secret-shared equality test, formalized as $\mathcal{F}_{\mathsf{ss\text{-}peqt}}$ in [4]. The $\mathsf{PlainAdd}$ operations introduce almost no overhead, while the secret-shared equality test is moderately more expensive and requires roughly $4\ell_{\mathsf{IS}} n_y$ bits of communication.

Comparison with DEL-free PSUs. Table 5 compares existing DEL-free PSU protocols with our proposal after applying the above modification. The results show that our DEL-free variant achieves state-of-the-art performance over prior DEL-free PSU protocols. More concretely, among the compared DEL-free PSU protocols, our protocol is the fastest in every tested setting, achieving up to $2.1\times$ speedup in the LAN setting and up to $9.8\times$ speedup in the WAN settings over the fastest prior DEL-free baseline. We note that [7], although highly efficient for standard semi-honest PSU, is not DEL-free and therefore is not included as a DEL-free baseline.

TABLE 5: Comparison of DEL-free PSU protocols. The item length is 64-bit for every case. All timing reports include setup timing, such as correlated random generation [27].

Set Size	Protocol	Comm. (MB)	Runtime (s)			
			10Gbps	1Gbps	100Mbps	10Mbps
2^{16}	[3]	97.46	0.94	39.51	44.08	84.54
	[4]	16.75	6.64	8.79	9.65	21.58
	Ours	17.26	0.53	2.46	3.85	16.48
2^{18}	[3]	387.71	3.75	46.84	55.58	337.20
	[4]	65.42	25.59	28.63	32.68	112.72
	Ours	48.51	1.82	4.27	7.31	43.14
2^{20}	[3]	1547.70	15.64	74.69	165.46	1346.76
	[4]	262.96	100.77	108.51	125.10	321.20
	Ours	164.72	7.32	11.02	21.10	144.15
2^{22}	[3]	6186.16	64.05	231.66	586.36	5453.89
	[4]	1062.79	407.05	433.95	501.18	1319.57
	Ours	591.74	30.51	40.13	76.48	517.69

Appendix E. Formal Proof for Theorem 4

Formal definition of semi-honest security. Consider a two-party protocol Π that computes an ideal functionality $f(X, Y)$ where party P_1 has input X and P_2 has input Y. For each party P_i, let $\mathsf{view}_i(X, Y)$ denote the view of P_i during an execution of Π on input X, Y. Specifically, it consists of the input X (or Y), the messages sent or received, the random tape sampled during the protocol execution, and the output of the protocol Π.

Definition 7. We say that a (two-party) protocol Π for P_1 and P_2 computes f securely against semi-honest

adversaries if there exist simulators Sim_1 and Sim_2 such that for any inputs X, Y of P_1 and P_2, respectively,

$$\mathsf{Sim}_1(X, f(X,Y)) \cong_c \mathsf{view}_1(X,Y)$$
$$\mathsf{Sim}_2(Y, f(X,Y)) \cong_c \mathsf{view}_2(X,Y)$$

where $\cong_c$ represents computational indistinguishability.

Simulation for the corrupted sender. In the ideal execution of $\mathcal{F}_{\mathsf{RPMT}}$, $\mathcal{S}$ has no output, and hence the simulator given only $\mathcal{S}$'s input Y. It behaves identically to the real protocol, except send a dummy ciphertext of a random vector $\tilde{\boldsymbol{p}}$, instead of the OKVS encoding of X'. This simulation is clearly indistinguishable from the real view, thanks to the IND-CPA security of RLWE.

Simulation for the corrupted receiver. In the ideal execution of $\mathcal{F}_{\mathsf{RPMT}}$, $\mathcal{R}$ is given the membership bit vectors $\boldsymbol{b} = (b_i)_{i\in[n_y]}$ where $b_i = \mathbf{1}(y_i \in X)$, and hence the simulator is given $\mathcal{R}$'s input X and the ideal output vector $\boldsymbol{b}$. The simulator is defined to run the protocol with the following fake OPRF set $\widetilde{Y}'$: Compute $F_k(X)$ by itself, randomly samples a $|X \cap Y|$-sized subset among $F_k(X)$, fill the set with a random element until reaching to size n_y, and finally reorder the fake set $\widetilde{Y}'$ so that $y'_i \in F_k(X)$ iff $b_i = 1$.

This simulation is indistinguishable from the real protocol's view. From $\mathcal{R}$'s point of view, $\mathcal{S}$-side protocol after OPRF depends on Y only through the randomized labels in Y'. For indices i with $b_i = 1$, all that matters for correctness is that the corresponding randomized value belongs to X', since every key in the OKVS table is associated with the same indicator value IS. For indices i with $b_i = 0$, the randomized values are pseudorandom labels outside X', and hence are computationally indistinguishable from fresh random labels. Therefore, conditioned on X' and the ideal output vector $\boldsymbol{b}$, the real randomized sender set Y' is computationally indistinguishable from the fake set $\widetilde{Y}'$.□

Appendix F. Full-field Random Band Widths

Methodology for random-band parameter choices of [6]. Given an input size n and an expansion ratio ε, the band width w is chosen so that the induced RB matrix $\boldsymbol{M} \in \mathbb{F}^{n\times m}$ is full rank with probability at least $1-2^{-\lambda}$. Typically λ is taken around 40, but it is infeasible to tune parameters while verifying a failure probability of 2^{-40}. To address this, [6] heuristically established the linear relation between

$$\lambda = a_\varepsilon \cdot w + g_{\varepsilon,n}, \tag{4}$$

whose core assumption is that the slope a_ε depends only on ε, but not on an input size n. Then, for a fixed ε, one measures the slope and intercept by varying n and w in a regime where the failure probability is feasibly measurable (e.g., $\lambda \le 15$), and extrapolates the fitted relation to determine the value of w for $\lambda = 40$.

Band-width comparison between binary-band and $\mathbb{Z}_p$-band. For our random spaced bands over $\mathbb{Z}_p$, we choose the band width w using the same full rank criterion as in [6]: for each expansion ratio ε, w is selected so that the induced matrix has full rank with probability at least $1-2^{-\lambda}$. Figure 13 compares the band width w required to achieve the target failure probability in the binary-band setting and in our $\mathbb{Z}_p$-band setting, and shows that the $\mathbb{Z}_p$-band choice consistently requires smaller w. Since [6] does not report binary-band parameters for all expansion ratios ε considered in our study, we re-implemented the binary-band baseline and ran the corresponding experiments over a range of ε.

Finally, to seek the parameters for our implementation Section 6, we follow the empirical fitting strategy of [6]: measuring failure probabilities $2^{-\lambda}$ for $\lambda < 15$ in the $\mathbb{Z}_p$-band setting and fit them to the linear model eq. (4). The results are summarized in Table 6

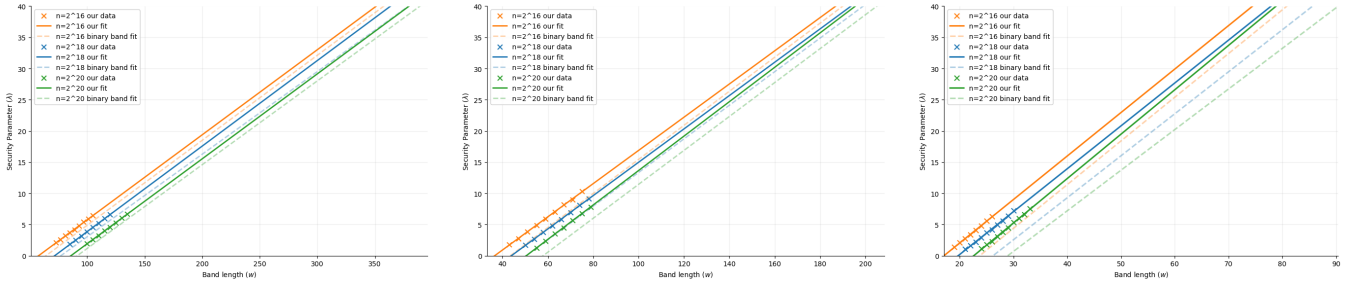
TABLE 6: Best-fit lines (4) for $\mathbb{Z}_p$-band, and the corresponding band width w for the encoding failure probability 2^{-40}, and the corresponding RPMT costs over LAN network.

$n = 2^{20}$				
ε	Fit line	w for $\lambda = 40$	Comm. (MB)	Runtime (s)
0.1	$0.2753w - 14.0200$	197	124.82	11.13
0.2	$0.4934w - 15.1968$	112	110	6.68
0.3	$0.6956w - 15.3069$	80	109	5.19
0.4	$0.7929w - 14.1219$	69	107.36	4.78
0.5	$0.9168w - 13.7755$	59	107.92	4.39
0.6	$1.1160w - 15.3274$	50	110.85	4.1
0.7	$1.3036w - 15.7176$	43	111.9	3.83
0.8	$1.3341w - 15.3561$	42	114.65	3.79
0.9	$1.5228w - 16.6124$	38	116.55	3.64
1.0	$1.7285w - 17.4118$	34	119.73	3.56
1.1	$1.9032w - 19.0024$	31	120.53	3.44
1.2	$1.9764w - 19.0728$	30	123.72	3.47
1.3	$1.9884w - 18.2284$	30	126.17	3.42
1.4	$2.1426w - 16.9244$	27	129.59	3.43
1.5	$2.1845w - 17.2011$	27	133.32	3.41
1.6	$2.2886w - 18.8276$	26	134.8	3.45
1.7	$2.3703w - 19.0591$	25	137.01	3.4
1.8	$2.5023w - 19.1985$	24	141.66	3.44
1.9	$2.6012w - 19.0890$	23	144.84	3.48
2.0	$2.6760w - 17.4406$	22	145.16	3.44

Appendix G. Asymptotic Cost Analysis

We explain here our RPMT protocol (Figure 6) has $O(n)$ computation and communication complexity in the input size n (where $n = n_x = n_y$). Since $\mathcal{F}_{\mathsf{oprf}}$ can each be realized with $O(n)$ communication and computation, it suffices to analyze the RLWE/OKVS components.

For the communication cost, $\mathcal{R}$ sends $b + w - 1$ RLWE ciphertexts to encrypt the OKVS encoding, where $b =$



(a) Failure probability for $\varepsilon = 0.05$ (b) Failure probability for $\varepsilon = 0.1$ (c) Failure probability for $\varepsilon = 0.3$

Figure 13: Comparison of the security parameter λ as a function of the band width w between our binary-band implementation and the $\mathbb{Z}_p$-band setting. Dashed lines denote the extrapolated linear fits from the binary-band measurements, while solid lines denote the extrapolated fits over $\mathbb{Z}_p$-band. For each $n \in \{2^{16}, 2^{18}, 2^{20}\}$, the solid curve lies to the left of the dashed curve, indicating that achieving the same target security level λ requires a smaller band width w in the $\mathbb{Z}_p$-band setting.

m/N; each has size $O(N)$, giving total communication $O((b+w-1)N) = O(n+Nw) = O(n)$, where the last equality holds because N is fixed and $w = O(\lambda)$. In the other direction, $\mathcal{S}$ sends $L = O(n/N)$ RLWE ciphertexts, equal to the number of matrices produced by the sequencing procedure, which is also $O(n)$ communication; recall from Section 4.2 for the reason that $L = O(n/N)$.

For the computation cost, we invoke RLWE.Enc exactly $b + w - 1$ times. Since each encryption call takes $O(N \log N)$ unit cost, the total encryption cost is $O((b+w-1)N \log N) = O((n+wN) \log N) = O(n)$. As the slot size N is a constant, we can say that this part takes $O(n)$ computational complexity. In RSB-OKVS.HomDecode, each sequenced layer matrix has at most $w + \zeta = O(w)$ diagonals—also recall from Section 4.2 for the reason that $w + \zeta = O(w)$—, there are $L = O(n/N)$ layers, and each homomorphic operation costs $O(N \log N)$ again; similar to the previous argument, the total homomorphic decoding cost is $O(nw)$. Since the band width satisfies $w = O(\lambda)$ for the statistical security parameter λ [6], this part is also linear in n. Finally, RLWE.Dec is invoked $L = O(n/N)$ times at cost $O(N \log N)$ each, for total decryption cost $O(n)$.

Appendix H. Communication Cost Analysis of [7]

Notation. Let $n_b = |X_b|$ denote each party's set size, $\tau = n_0 + n_1$, and $u = |X_0 \cup X_1| \le \tau$. The Invertible Bloom Lookup Table (IBLT) is parameterized by the number of hash functions k, the subtable length ℓ, and the total number of bins $m = k\ell \simeq e\tau$, where e is the IBLT expansion factor. Let κ denote the computational security parameter. We write L for the bit-length of the encoded value carried in the IBLT sum field and in the UnionPeel messages, including the encoding overhead for the special symbol $\perp$. For exact PSU over arbitrary a-bit items under the direct one-to-one encoding assumed in [7], one has $L \ge a$ up to this encoding overhead.

Per-bin payload. The protocol Π_{PSU} of [7, Fig. 5] repeatedly invokes the sub-protocol $\Pi_{\mathrm{UnionPeel}}$ of [7, Fig. 4] on a public bin set $Q_\pi^{(t)}$. In the optimized version of [7], the per-bin OPRF calls are replaced by a one-time multi-point OPRF on X_1 during setup. We therefore exclude this setup term and count only the repeated per-bin OT/reveal payload.

For each examined bin, UnionPeel communicates

$$c_{\mathrm{bin}}(L) = \underbrace{2L}_{\text{1-out-of-2 OT}} + \underbrace{3\max(L,\kappa)}_{\text{1-out-of-3 OT}} + \underbrace{L}_{\substack{\text{reveal} \\ P_1 \to P_0}} = 3L + 3\max(L,\kappa) \quad \text{bits.} \tag{5}$$

The 1-out-of-3 OT term uses standard bit-string OT extension: its mixed-domain messages are encoded as bit strings of common length $\max(L,\kappa)$. We also assume that the public bin set $Q_\pi^{(t)}$ is enumerated in a fixed common order, so the bin index (i,j) need not be retransmitted. If indices are sent explicitly, an additional $\lceil \log_2 k \rceil + \lceil \log_2 \ell \rceil$ bits per examined bin should be added.

Total number of bin evaluations. Let $q_t = |Q_\pi^{(t)}|$. From $Q_\pi^{(0)} = [k] \times [\ell]$ and $Q_\pi^{(t+1)} = \{(i, h_i(v)) : i \in [k], v \in V_\pi^{(t)}\}$ in [7, Fig. 5], we have $q_0 = m$ and $q_{t+1} \le k|V_\pi^{(t)}|$. Since the recovered elements $\bigcup_t V_\pi^{(t)}$ form $X_0 \cup X_1$ on a successful execution,

$$Q_\Sigma := \sum_{t \ge 0} q_t \le m + k|V_\pi^{(t)}| \le e\tau + k\tau = (e+k)\tau. \tag{6}$$

Closed-form bound. Combining (5) and (6), the dominant item-length-dependent payload satisfies

$$C_{\mathrm{payload}}(L) \le (e+k)\tau\big(3L + 3\max(L,\kappa)\big) \quad \text{bits.} \tag{7}$$

For $\kappa = 128$, the per-bin payload factor is piecewise-linear:

$$c_{\mathrm{bin}}(L) = \begin{cases} 3L + 384, & L \le 128, \\ 6L, & L > 128. \end{cases} \tag{8}$$

Thus

$$C_{\mathrm{payload}}(L) \le (e+k)\tau \cdot c_{\mathrm{bin}}(L),$$

and the slope doubles at $L = \kappa$ because the 1-out-of-3 OT payload also becomes L-dominated.

Normalizing by the 64-bit baseline used in [7, Table 2] eliminates the dependence on τ, e, and k:

$$\frac{C_{\mathrm{payload}}(L)}{C_{\mathrm{payload}}(64)} = \frac{L + \max(L, 128)}{192}. \tag{9}$$

This gives the ratios $1.33\times$, $2.67\times$, and $5.33\times$ at $L = 128$, 256, and 512 bits, respectively.